UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

VIETNAM FRANCE UNIVERSITY

BACHELOR THESIS

Deep learning-based Automated Tennis Tracking System for Line Calling

Student: Tran Ngoc Hung
Student ID: 23BI14181
Major: Data Science
Intake: 2023–2026

Supervisor: Assoc. Prof. Nguyen Duc Dung

HANOI, 2026

Chapter 1

Introduction

1.1 Background and Motivation

Motion in modern sport is fast and highly variable, which makes consistent manual measurement difficult. As recording has spread from broadcast crews to ordinary phones and consumer cameras, computer vision (CV) has become a practical tool for sports analytics, applied to problems from player and ball tracking through to tactical and performance analysis [1]. Tennis is a representative and demanding case: matches are decided by the trajectory of a small, rapidly moving ball over a precisely measured court, so an accurate spatial and temporal understanding of both the ball and the playing surface is fundamental to almost any downstream analysis.

The commercial prominence of vision-based officiating illustrates both the value and the difficulty of this problem. Systems such as Hawk-Eye are now embedded in professional tennis for automated line-calling, yet scholarly analysis has cautioned that their confident, deterministic visualisations can obscure the measurement uncertainty inherent in any reconstruction of ball position [2]. Such systems also depend on calibrated multi-camera installations that are unavailable outside elite venues. This motivates a complementary line of work: vision-only analysis of ordinary monocular video that is reproducible, inexpensive, and accessible for coaching, amateur play, and research.

Automated tennis analysis from a single video stream rests on three perception subproblems. The first is ball tracking: localising the ball in every frame. This is a canonical small-object problem, because the ball occupies only a handful of pixels, moves quickly enough to blur or leave afterimages, and is frequently occluded by players, the net, or court markings, so that single-frame appearance is often insufficient. Heatmap-based temporal models such as TrackNet were introduced precisely to exploit motion cues across consecutive frames in this setting [3], in contrast to single-frame object detectors of the YOLO family that localise objects through bounding-box regression [4]. The second subproblem is court keypoint detection and registration: recovering the geometric mapping between image pixels and real-world court coordinates, typically by detecting a fixed set of court landmarks and fitting a homography with a robust estimator such as random sample consensus (RANSAC) [5]. Keypoint localisation has been tackled both with high-capacity backbones such as deep residual networks [6] and high-resolution pose networks [7], and with efficient mobile architectures designed for constrained hardware [8]. The third subproblem is bounce-event detection: identifying the discrete frames at which the ball contacts the court, recovered from a noisy and intermittently missing trajectory. This is a temporal pattern-recognition task that can be addressed with hand-crafted kinematic

heuristics, gradient-boosted decision trees [9], or temporal convolutional networks (TCNs) [10].

A wide range of models has been proposed for each of these sub-problems individually. However, they are commonly reported on heterogeneous datasets and private train/test splits, and most studies evaluate a single proposed model rather than placing competing designs on a common footing. As a result, it is difficult to draw like-for-like conclusions about the accuracy, speed, and robustness trade-offs between architectures within a sub-task, and, to the best of the author's knowledge, comparatively few works compose the resulting components into a single, end-to-end pipeline whose practical deployability can be assessed. The present thesis is motivated by both gaps: the scarcity of controlled within-task comparison, and the still rarer attempt to assemble the resulting components into one reproducible pipeline.

1.2 Problem Statement

This thesis studies the three perception sub-tasks of monocular tennis video analysis (ball tracking, court keypoint detection with homography, and bounce-event detection), together with the engineering problem of integrating them into a single pipeline. Each sub-task is framed as a controlled model comparison conducted under a shared, deterministic evaluation protocol.

The sub-tasks are formulated as follows. Ball tracking is the per-frame estimation of the ball's image position, with an explicit "not visible" state for frames in which the ball is absent or undetectable. Court detection is the per-frame localisation of a fixed set of fourteen predefined court keypoints, followed by estimation of a homography that maps the image onto a canonical court template expressed in metric units. Bounce detection is the classification of which frames correspond to a ball bounce, distinguished both from racket-hit frames and from the much larger set of ordinary in-flight frames.

A central methodological point is that this work draws on two distinct datasets. Ball tracking and bounce detection share the publicly available TrackNet tennis dataset, in which each frame is annotated with ball visibility, image coordinates, and a status label. Court keypoint detection uses a separate court-keypoint dataset with its own annotation schema and image resolution. Because the two datasets differ in content, resolution, and split policy, accuracy figures are comparable only within a sub-task and never across sub-tasks; this constraint is stated explicitly and respected throughout the thesis. The integration sub-problem is distinct from the three perception problems: composing the strongest model from each sub-task into one system that ingests raw video and emits a ball trajectory, court geometry, and bounce events raises questions of model coupling, export and packaging, and end-to-end throughput that do not arise when models are evaluated in isolation. The purpose of this integrated system is to support automated line calling: by composing the court geometry with the detected bounce events, the pipeline determines, for each bounce, whether the ball landed inside or outside the court boundary. This in/out decision is obtained geometrically, by projecting the bounce position through the estimated homography into court-metric coordinates and testing for containment within the court-boundary polygon; it is qualitative, because no ground-truth line-call labels are available in either dataset.

1.3 Objectives and Research Questions

The overall objective of this thesis is to design, implement, and rigorously compare representative models for each sub-task on a common protocol, and to demonstrate that the strongest models can be integrated into a deployable pipeline that supports automated line calling. Concretely, the work aims to: (i) construct a deterministic, reproducible benchmark for each sub-task; (ii) train and evaluate competing models under a shared metric suite; (iii) integrate the best-performing models into one end-to-end system that determines, for each detected bounce, whether the ball landed inside or outside the court (an automated in/out line call); and (iv) release the artifacts required to reproduce the results. This objective is articulated through four research questions (RQs).

RQ1 (Ball tracking).
Among the TrackNet family (V1–V5) and a single-frame detector (YOLO11m), which most accurately localises a small, fast, and frequently occluded tennis ball, and what does the architectural progression buy in accuracy relative to inference speed?

RQ2 (Court detection).
Which backbone offers the best accuracy/latency trade-off for fourteen-keypoint court detection among a heatmap convolutional neural network (CNN), a deep residual network, a high-resolution network, and a lightweight mobile network, and is the resulting keypoint accuracy sufficient to drive a reliable homography to court-metric space?

RQ3 (Bounce detection).
For bounce-event timing from a noisy ball trajectory, how do a hand-built heuristic, a gradient-boosted per-frame classifier, and a temporal convolutional network compare in accuracy, and what does each cost in engineering effort?

RQ4 (Integration).
Can the strongest model from each sub-task be composed into a single pipeline that ingests raw video and outputs a tracked ball, court geometry, bounce events, and, as its culminating output, a per-bounce in/out line-call decision obtained by projecting each bounce through the court homography, demonstrating the line-calling capability as a qualitative geometric result rather than a validated accuracy figure, and what throughput and deployment trade-offs (model export, per-frame latency, and processing bottlenecks) does such a composition incur?

1.4 Contributions

In addressing these questions, the thesis makes the following contributions.

1. A deterministic, reproducible benchmark protocol for each sub-task, comprising a fixed data-split contract and a defined metric suite per task (a per-clip split of the TrackNet dataset for ball tracking and bounce detection, and a deterministic hash-based split of the court-keypoint dataset for court detection), enabling controlled, like-for-like comparison within each sub-task.

2. Building on that protocol, a systematic comparison of six ball-tracking, four court-detection, and three bounce-detection models, each evaluated on the dataset appropriate to its task, in contrast to the common practice of reporting only a single proposed model.

3. An integrated, packaged pipeline that wires the strongest model from each sub-task into one system, with neural-network export to the Open Neural Network Exchange (ONNX) format and an interactive demonstration interface, illustrating practical deployability. By combining the estimated court homography with the detected bounces, the pipeline further produces an automated in/out line-call decision for each bounce, demonstrating the line-calling application that names the thesis.

4. Finally, the reproducibility artifacts themselves (split manifests, training configurations, metric outputs, and visualisations), collected in a single repository.

These contributions are framed as a controlled comparative study on two public-style datasets rather than as a claim of state-of-the-art performance on a competition benchmark; the scope and its limitations are stated in Section 1.5 and revisited in Chapter **??**.

1.5 Scope and Limitations

The scope of this thesis is deliberately bounded. The study concerns offline analysis of recorded monocular tennis video; it does not target live, real-time operation, and the measured per-frame latency of the integrated pipeline on commodity CPU hardware does not support a real-time claim (Chapter **??**). The evaluation rests on two datasets (one for ball and bounce, one for court), so the conclusions pertain to the conditions those datasets represent and may not transfer without adaptation to markedly different camera angles, court surfaces, or broadcast styles. The bounce-detection test set in particular contains relatively few bounce events, so the corresponding figures should be read with that sample size in mind. The in/out line-call output is likewise a qualitative geometric demonstration: the available datasets provide no ground-truth line-call labels, and the reprojection and bounce-localisation uncertainties are not propagated into a calibrated confidence, so the in/out determination is not quantitatively validated and the system is not presented as a calibrated, officiating-grade line-calling instrument, consistent with the cautionary framing of Section 1.1 [2]. Finally, the system addresses ball, court, and bounce perception only: it does not perform player tracking, pose estimation, shot classification, multi-camera fusion, or three-dimensional trajectory reconstruction. These omissions define directions for future work (Chapter **??**).

1.6 Thesis Organisation

The remainder of this thesis is organised as follows. Chapter 2 reviews background and related work across small-object detection, ball tracking, court registration, and trajectory-based event detection, and situates the research gap. Chapter 3 describes the two datasets, the split policies, and the shared metric vocabulary. Chapter 4 presents the ball-tracking comparison; Chapter 5 the court keypoint detection and homography pipeline; and Chapter 6 the bounce-event detection comparison. Chapter **??** describes the integrated system, its model-selection rationale, and its deployment characteristics.

Chapter ?? discusses cross-cutting findings, threats to validity, and reproducibility, and Chapter ?? concludes by answering the research questions and outlining future work.

Chapter 2

Background and Related Work

This chapter reviews the research that supports the three perception sub-tasks studied in this thesis, together with the work on integrated analysis systems that motivates their composition. The review is organised thematically rather than paper by paper. Section 2.1 establishes the two dominant models for localising objects in images, bounding-box regression and heatmap regression, and explains why the latter is better suited to small, fast targets. Section 2.2 surveys ball tracking in sports video, with particular attention to the TrackNet family that this thesis benchmarks. Section 2.3 covers court and field registration, spanning keypoint-detection backbones and the geometry of homography estimation. Section 2.4 reviews event detection from trajectories, including temporal convolutional networks and gradient-boosted decision trees. Section 2.5 situates the work within integrated tennis systems and automated officiating, and Section 2.6 synthesises the preceding material into the research gap that this thesis addresses.

2.1 Object Detection and Small-Object Localisation

2.1.1 Bounding-box detection

Modern object detection was reshaped by the region-based convolutional neural network (R-CNN), which combined bottom-up region proposals with convolutional features and demonstrated a large accuracy gain over methods built on hand-crafted features [11]. The original design was computationally expensive because it processed every proposal independently; Fast R-CNN removed this redundancy by sharing one convolutional pass over the whole image and pooling features per region [12], and Faster R-CNN replaced the external proposal step with a learned region proposal network, yielding a near end-to-end two-stage detector [13]. In parallel, one-stage detectors removed the proposal stage altogether: the single shot multibox detector (SSD) predicted boxes directly from multiple feature maps [14], while the You Only Look Once (YOLO) family framed detection as a single regression over a spatial grid [4]. This last design accepted a modest accuracy reduction in return for substantially higher throughput. Successive YOLO revisions refined the backbone and the multi-scale prediction head [15], and the line has since continued through widely used engineering releases that, at the time of writing, are documented as software rather than peer-reviewed publications [16]. Two architectural ideas from this lineage are especially relevant to small objects. Feature pyramid networks (FPNs) build a top-down feature hierarchy with lateral connections so that small ob-

jects remain represented at high spatial resolution [17], and the focal loss addresses the extreme foreground-background imbalance that otherwise overwhelms dense one-stage training [18].

Despite these advances, detecting small objects remains difficult. A recent survey identifies the loss of spatial detail through repeated downsampling, the scarcity of discriminative appearance cues, and the imbalance between tiny foreground regions and large backgrounds as persistent obstacles [19]. A tennis ball is an acute instance of this problem. It occupies only a few pixels and is frequently blurred by motion, and because its bounding box is barely larger than a point, the box-regression objective conveys little information beyond a location.

2.1.2 Heatmap regression

An alternative to box regression represents a target as a spatial probability map, in which a Gaussian peak marks the object centre and the predicted coordinate is recovered by locating the maximum response. This formulation originated in human pose estimation, where convolutional pose machines refined per-joint belief maps through successive stages [20] and stacked hourglass networks repeatedly pooled and upsampled features to aggregate evidence across scales [21]. The idea was later generalised to object detection by representing each object as a single centre point in a heatmap, which removed the need for anchor boxes and non-maximum suppression [22]. Heatmap regression is attractive for small, fast targets for several reasons. It supplies dense per-pixel supervision, and it localises a target sub-pixel through the shape of the response rather than through a discrete box. It also accommodates the absence of a target naturally, as a map with no significant peak. These properties make it the dominant formulation for both the ball-tracking and court-keypoint sub-tasks studied in this thesis, and they motivate comparing heatmap-based trackers with a single bounding-box detector when those sub-tasks are taken up in later chapters.

2.2 Ball Tracking in Sports Video

2.2.1 Tracking by detection

A common strategy for following an object through a video is tracking by detection, in which a per-frame detector is coupled with a temporal data association step. Simple online and realtime tracking (SORT) associates detections across frames using a Kalman-filter motion model and the Hungarian assignment algorithm, achieving high speed with bounding-box overlap as the sole association cue [23], and DeepSORT augments this with a learned appearance descriptor to reduce identity switches through occlusion [24]. These methods are effective for pedestrians and vehicles, but they rest on assumptions that a tennis ball violates: the ball has almost no distinctive appearance to embed, it moves fast enough to blur or skip across the frame, and it is regularly occluded by players, the net, and court lines. Single-frame detection followed by geometric association is therefore fragile in this setting, which has motivated trackers that reason over several frames jointly.

2.2.2 The TrackNet family

TrackNet introduced a heatmap-based, multi-frame approach designed specifically for small and fast sports objects [3]. Its encoder-decoder network ingests three consecutive frames, stacked into a nine-channel input, and outputs a Gaussian heatmap locating the ball, so that motion across the input frames provides the temporal cue absent from a single image. This original design follows a multiple-in single-out pattern, in which several input frames yield one output heatmap. TrackNetV2 reorganised the network into a multiple-in multiple-out architecture that predicts a heatmap for every input frame in one pass, reducing redundant computation and improving throughput while retaining accuracy on shuttlecock data [25]. TrackNetV3 added two refinement components on top of this backbone: an auxiliary background-aware module to improve robustness under visual interference, and an inpainting-based trajectory-rectification module that repairs missed detections by reasoning over the predicted path [26]. TrackNetV4 introduced a lightweight motion-attention mechanism, in which inter-frame difference maps are fused with the backbone features so that the network explicitly attends to moving-ball regions, with reported gains under occlusion and low visibility [27]. The lineage continues to be extended through further variants proposed as preprints [28], but this thesis treats the published members of the family as its primary reference points.

Beyond the TrackNet line, related work has tackled ball and event analysis in adjacent racket and table sports. TTNet performs joint ball detection, semantic segmentation, and temporal event spotting for table tennis within a single multi-task network [29]. This shows that detection and event recognition can share a representation. Patch-wise convolutional classifiers have also been applied to tennis ball detection in broadcast video, linking local detections into trajectories [30]. Taken together, this literature establishes multi-frame heatmap regression as the prevailing paradigm for localising a small, fast ball, and frames the comparison undertaken in Chapter 4 between successive members of the TrackNet family and a representative single-frame detector.

2.3 Court Keypoint Detection and Geometric Registration

2.3.1 Keypoint detection backbones

Recovering the geometry of the playing surface begins with localising a fixed set of court landmarks, a task closely related to human-pose keypoint estimation. The heatmap-regression backbones reviewed in Section 2.1.2 transfer directly to this problem. A widely used baseline appends a few deconvolutional layers to a deep residual network (ResNet) backbone [6] to produce keypoint heatmaps from high-level features, a design valued for its simplicity [31]. In contrast, the high-resolution network (HRNet) maintains high-resolution representations throughout the network and fuses parallel multi-resolution streams, which improves the spatial precision of the predicted keypoints [7]. Because court detection must often run within a larger real-time pipeline, efficient backbones are also of interest: the MobileNet family uses inverted-residual blocks with linear bottlenecks [32] and architecture search with squeeze-and-excitation refinements [8] to deliver competitive accuracy at a small fraction of the parameter count. Whether such lightweight backbones suffice for accurate court-keypoint localisation is one of the open

questions this thesis sets out to answer.

2.3.2 Sports-field registration and homography

Localised keypoints are useful for analysis only once they are related to a metric model of the court. For a planar surface viewed by a single camera, this relationship is a homography, a projective transformation between the image plane and the court plane whose theory is set out in the standard reference on multiple-view geometry [33]. Estimating the homography from automatically detected keypoints requires robustness to mislocalised points, which is classically provided by random sample consensus (RANSAC), an iterative scheme that fits a model to randomly sampled minimal subsets and retains the hypothesis with the largest consensus set [5]. Learned variants such as differentiable RANSAC make the hypothesis-selection step trainable end to end [34]. Within sports specifically, field registration has been framed as inference in a deep structured model that combines semantic cues with geometric priors [35], as iterative optimisation through a learned error function [36], and as dense keypoint detection on a regular grid coupled with feature-warp refinement to remain robust where distinctive corners are absent [37]. The court pipeline in Chapter 5 follows the detect-then-fit pattern common to this literature: predict court keypoints with a heatmap backbone, then fit a homography with a robust estimator to map the image into court-metric coordinates.

2.4 Event Detection from Trajectories

2.4.1 Temporal modelling of sequences

Identifying the discrete frames at which a bounce occurs is a temporal pattern-recognition problem over a one-dimensional signal derived from the ball trajectory. Although recurrent networks were long the default for sequence modelling, convolutional alternatives have proved competitive. Dilated causal convolutions, popularised for raw-audio generation by WaveNet, expand the receptive field exponentially with depth without increasing the parameter count [38]. Long temporal contexts can therefore be captured efficiently. Temporal convolutional networks (TCNs) build on this principle for discriminative tasks: they have been applied to fine-grained action segmentation and detection in video [39], and a systematic comparison found that generic TCNs match or exceed recurrent networks across a range of sequence-modelling benchmarks while being simpler to train [10]. These properties make a dilated TCN a natural candidate for classifying windows of trajectory features into bounce events.

2.4.2 Gradient-boosted decision trees

A problem is naturally expressed through engineered per-frame features, gradient-boosted decision trees offer a strong and data-efficient alternative to deep models. Gradient boosting fits an additive ensemble of trees by successively approximating the negative gradient of a loss function [40]. Modern implementations differ mainly in how they control overfitting and scale to large data: XGBoost introduced a regularised objective and a scalable system design [9], LightGBM accelerated training through histogram-based splits and feature bundling [41], and CatBoost reduced the prediction bias of ordinary boosting through ordered boosting [42]. Histogram-based gradient boosting is also available as

a standard component of general machine-learning toolkits [43]. Such models operate directly on the kinematic and shape features computed from a trajectory and require comparatively little tuning, which makes a gradient-boosted classifier a natural baseline among the bounce detectors this thesis compares.

2.4.3 Bounce and hit detection in tennis

Compared with ball localisation, the detection of bounce and hit events from a tennis trajectory has received comparatively little dedicated attention, and existing approaches range from hand-built kinematic heuristics that look for characteristic reversals in vertical motion to learned classifiers over trajectory features. Multi-task systems such as TTNet show that event spotting can be learned jointly with detection [29], but such work generally targets a different sport and a different event vocabulary. The relative scarcity of controlled comparisons among heuristic, tree-based, and temporal-convolutional detectors for tennis bounces, evaluated on a common trajectory under a common tolerance, is one of the gaps that Chapter 6 addresses.

2.5 Integrated Tennis Systems and Automated Officiating

The most prominent application of tennis computer vision is automated officiating. The Hawk-Eye system reconstructs the three-dimensional ball trajectory from several synchronised, calibrated cameras and estimates the bounce position to adjudicate line calls [44]. Its commercial success demonstrates the value of vision-based measurement, yet scholarly analysis has cautioned that the confident, deterministic visualisations such systems present can mask the measurement uncertainty inherent in any reconstruction of ball position [2]. Equally important for the present work, these systems depend on calibrated multi-camera installations that are unavailable outside elite venues, which motivates accessible, vision-only analysis of ordinary monocular video.

More broadly, surveys of computer vision for sport document a wide range of perception and analysis tasks, from player and ball tracking to tactical understanding, and note that components are typically developed and reported in isolation [1, 45]. The literature contains strong individual models for ball tracking, court registration, and event detection, but comparatively few studies compose these components into a single monocular pipeline whose end-to-end behaviour and deployability can be examined, which is precisely the integration studied in Chapter **??**.

2.6 Research Gap and Positioning

Three observations emerge from the preceding review. First, within each of the three perception sub-tasks, a wide range of architectures has been proposed, but they are commonly reported on heterogeneous datasets and private train/test splits, and most studies evaluate a single proposed model rather than placing competing designs on a common footing. This makes it difficult to draw like-for-like conclusions about the accuracy, speed, and robustness trade-offs between architectures within a sub-task. Second, the heatmap formulation that recurs across ball tracking and court detection, and the temporal and

tree-based models that recur across event detection, are seldom compared head to head under a fixed protocol on the same data. Third, and to the best of the author's knowledge, comparatively few works compose the resulting components into a single end-to-end monocular pipeline whose practical deployability, including model export and per-frame latency, is assessed, and fewer still close the loop to a geometric line-call decision from ordinary single-camera video.

This thesis is positioned against these gaps. It conducts a controlled, within-task comparison for each of the three sub-tasks under a deterministic evaluation protocol, and it composes the strongest model from each into one reproducible pipeline that maps detected bounces through an estimated homography to a qualitative in/out line call. The protocol, datasets, and shared metric vocabulary that make these comparisons reproducible are described next, in Chapter 3.

Chapter 3

Datasets and Evaluation Methodology

This chapter sets out the experimental foundation shared by the three perception subtasks studied in the thesis. Because the ball, court, and bounce comparisons each reuse the same datasets, split contracts, and metric definitions, these elements are described once here rather than repeated in every results chapter. Section 3.1 describes the TrackNet tennis dataset, which underlies both ball tracking and bounce detection, and Section 3.2 describes the separate court-keypoint dataset used for court detection. Section 3.3 states explicitly that the two datasets are not cross-comparable. Section 3.4 defines the deterministic split policies, and Section 3.5 defines the shared metric vocabulary. Task-specific preprocessing, learning targets, and training details are not covered here: they belong with the models that consume them and are presented in Chapters 4, 5, and 6 respectively.

3.1 The TrackNet Tennis Dataset

Ball tracking and bounce detection are both evaluated on the publicly available tennis dataset released with the original TrackNet work [3]. The dataset is drawn from broadcast tennis video and is organised into ten matches, named game1 through game10. Each match is divided into short clips of consecutive frames, with one clip per folder containing the extracted JPEG frames and a single annotation file, Label.csv. In the copy used in this work the dataset comprises 95 clips and 19,835 annotated frames in total; its composition per match is given in Table 3.1.

Table 3.1: Composition of the TrackNet tennis dataset by match, as used in this thesis. Both the ball-tracking and bounce-detection comparisons draw from this same pool of frames.

Match	Clips	Frames
game1	13	1,978
game2	8	2,222
game3	9	1,888
game4	7	2,238
game5	15	1,964
game6	4	1,989
game7	9	1,881
game8	9	1,958
game9	9	1,573
game10	12	2,144
Total	95	19,835

3.1.1 Annotation schema

Every row of a "Label.csv" file annotates one frame with five fields: the frame file name, a visibility class, the ball x-coordinate and y-coordinate in image pixels, and a status label. The visibility field follows the dataset's four-level convention: class 0 marks frames in which the ball is outside the frame or otherwise not visible; class 1 marks frames in which the ball is easily identifiable; class 2 marks frames in which the ball is present but difficult to identify, for example under motion blur; and class 3 marks frames in which the ball is occluded by another object. The status field records, independently of visibility, whether a salient event occurs at that frame: status 0 denotes ordinary in-flight frames with no event, status 1 a racket hit, and status 2 a ball bounce on the court. Because a hit or a bounce is defined by how the ball moves rather than by its appearance in a single frame, Figure 3.2 illustrates each status value with the ball trajectory over a short window around an example event, while Figure 3.1 illustrates each visibility class with a representative frame and a magnified view of the ball region. The distribution of both labels across the full dataset is summarised in Table 3.2. The ball is clearly visible in roughly 89% of frames, while bounce events, the target of Chapter 6, account for only about 2.6% of frames, a sparsity that directly shapes the choice of evaluation metric for that task (Section 3.5.3).

Throughout the pipeline, a single encoding of the absence of a ball is used: when an annotation carries visibility 0, or its coordinate fields are blank or negative, the ball position is represented as $(-1, -1)$. This convention is preserved by every dataset loader, model output, and metric in both the ball and bounce projects, so that "no ball" is always distinguishable from a coordinate that happens to lie near the image origin.

3.2 The Court-Keypoint Dataset

Court keypoint detection is evaluated on a separate, publicly distributed dataset, the TennisCourtDetector collection [46], which is unrelated to the TrackNet tennis data of

Table 3.2: Label distribution across all 19,835 frames of the TrackNet dataset. Visibility and status are annotated independently of one another.

Field	Class	Frames	Share
Visibility	0: not visible	729	3.7%
	1: easily visible	17,632	88.9%
	2: hard to identify	1,392	7.0%
	3: occluded	82	0.4%
Status	0: in flight (none)	18,796	94.8%
	1: hit	516	2.6%
	2: bounce	523	2.6%

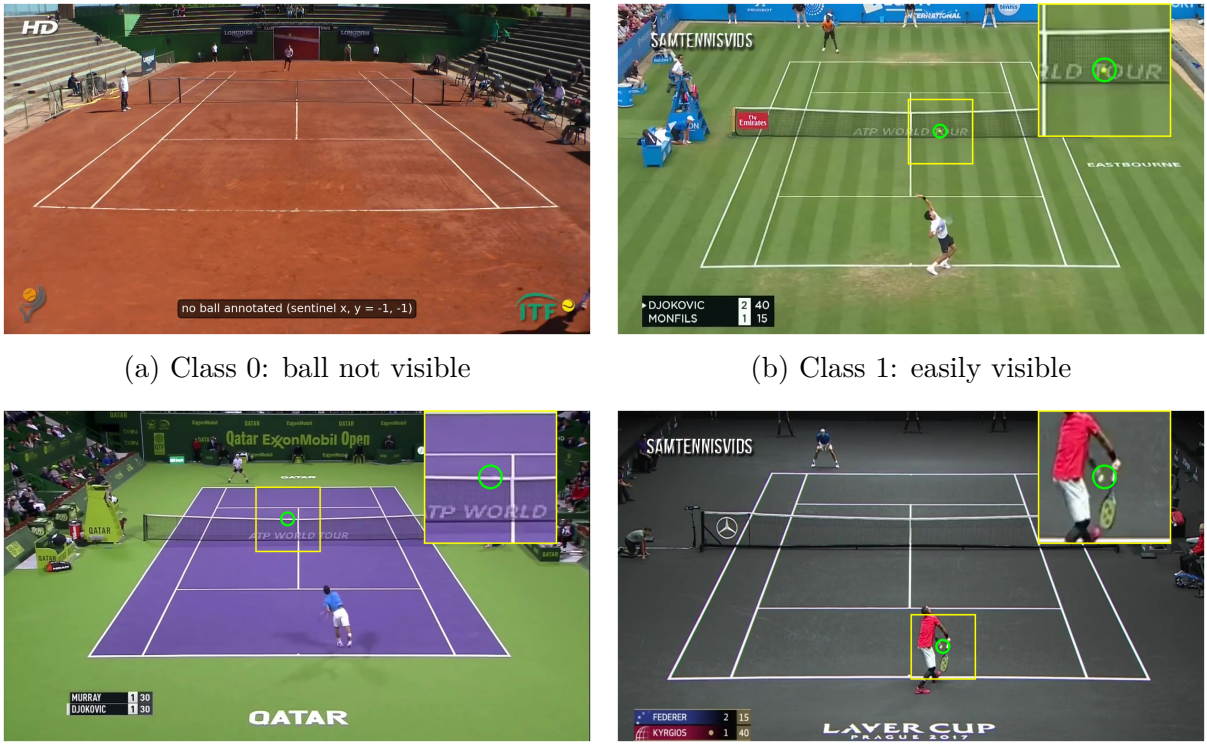
Section 3.1. It consists of 8,841 single RGB frames at an original resolution of 1280 × 720 pixels, supplied as "images/{record_id}.png", together with two JSON annotation files: "data_train.json" with 6,630 records and "data_val.json" with 2,211 records. Each record provides an identifier and a list of fourteen $[x, y]$ keypoints marking fixed court landmarks such as the doubles and singles corners, the service points, and the centre service-line intersections. Figure 3.3 shows two annotated frames with these fourteen keypoints overlaid. The detailed ordering of these fourteen keypoints, the construction of an inferred court-centre point from the outer-court diagonals, and the canonical court template used for geometric registration are specific to the court pipeline and are therefore deferred to Chapter 5; only the dataset's overall size and split, needed to define the protocol, are stated here.

3.3 Non-Comparability of the Two Datasets

A point of method that recurs throughout the thesis is stated here once for clarity. Ball tracking and bounce detection operate on the TrackNet tennis dataset, whereas court detection operates on the TennisCourtDetector dataset. The two differ in their content (a small moving ball against broadcast backgrounds versus static court geometry), in their image resolution and source, in their annotation schema (per-frame CSV labels versus per-image JSON keypoints), and in their split policies (Section 3.4). Because of these differences, accuracy figures are meaningful only within a sub-task and must never be compared across sub-tasks. A pixel-accuracy figure for the ball and a pixel-accuracy figure for court keypoints, for instance, are measured on different images at different resolutions and do not constitute a like-for-like comparison. The benchmark of this thesis is therefore unified within each sub-task rather than across the three.

3.4 Split Policies

All evaluation in this thesis rests on fixed, deterministic train, validation, and test splits, so that every model within a sub-task is trained and judged on exactly the same frames and the results are reproducible from a recorded seed. The two datasets follow distinct split contracts, which are described in turn and summarised together in Table 3.3.



(a) Class 0: ball not visible

(b) Class 1: easily visible

(c) Class 2: hard to identify

(d) Class 3: occluded

Figure 3.1: Representative frames for the four TrackNet visibility classes. In each panel the ground-truth ball position is marked with a green circle and magnified in the yellow inset; class 0 carries no ball annotation. The examples are drawn from different matches and court surfaces (clay, grass, and hard court), which also illustrates the visual variety of the dataset.

3.4.1 TrackNet splits for ball and bounce

Both TrackNet-based sub-tasks split the data by clip rather than by frame, so that frames from one clip never appear in more than one split. This is essential: consecutive frames within a clip are highly correlated, and allowing them to straddle the train and test boundaries would leak information and inflate the apparent accuracy. Within this shared principle, however, the two sub-tasks adopt different partitions of the same 19,835-frame pool, reflecting their different goals.

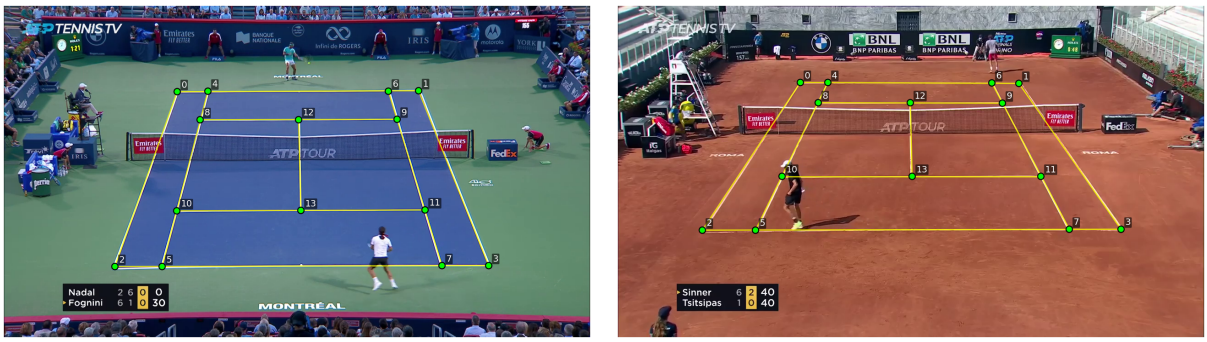
For ball tracking, matches game1 through game9 are used for training and validation, and game10 is held out in its entirety as an unseen test match. Within the training matches, the clips of each match are shuffled with a fixed seed (42) and partitioned into 85% training and 15% validation. This yields 14,812 training frames in 73 clips, 2,879 validation frames in 10 clips, and 2,144 test frames in 12 clips.

For bounce detection, a separate manifest holds out a different match, game7, as the test set, because the goal is to measure event timing on a match whose bounce patterns were never seen in training. The remaining matches are partitioned into training and validation clips stratified by bounce density, so that the scarce bounce events are represented in both partitions. This yields 14,697 training frames in 71 clips (387 bounce events), 3,257 validation frames in 15 clips (86 bounce events), and 1,881 test frames in 9 clips (50 bounce events).

The consequence, made explicit in Table 3.3, is that even the two TrackNet sub-tasks

(a) Status 0: in flight (b) Status 1: hit (c) Status 2: bounce

Figure 3.2: The three status values, each shown through the ball trajectory over a window of fifteen frames centred on an example event. Points are coloured by time (dark to bright), and the green ring marks the labelled frame. An in-flight frame lies on a smooth arc, a hit occurs where the ball is struck at the player's racket, and a bounce occurs where the ball lands on the court. These cues are temporal, which is why bounce detection (Chapter 6) operates on the trajectory rather than on single frames.



(a) Hard court (b) Clay court

Figure 3.3: Two example frames from the court-keypoint dataset with their ground-truth annotation overlaid. The fourteen annotated keypoints are shown as green markers numbered in the dataset ordering, and the yellow lines are the court boundaries they imply. The two frames show different camera angles and court surfaces, illustrating the variation the court models must handle. The meaning of each keypoint index is given in Chapter 5.

draw their test sets from different matches (game10 for the ball, game7 for bounces) and partition the remaining data by different rules. Their figures are therefore reported against different held-out data and are not interchangeable, a finer instance of the non-comparability principle of Section 3.3.

3.4.2 Court split

The court-keypoint dataset is split deterministically without a random seed. The 6,630 records of "data_train.json" are used as the training set as supplied. The 2,211 records of "data_val.json" are ordered by the MD5 hash of their record identifier and divided in half: the first 1,105 records form the validation set and the remaining records form the test set. Six test records carrying semantically incorrect annotations are excluded, leaving 1,100 test records.

Table 3.3: Split allocations for the three sub-tasks. The two TrackNet sub-tasks (ball, bounce) share the same 19,835-frame pool but use different held-out test matches and different partition rules; the court sub-task uses a separate dataset of image records, so its counts are not comparable with the TrackNet columns.

	Ball (TrackNet)		Bounce (TrackNet)			Court
Split	Frames	Clips	Frames	Clips	Bounces	Records
Train	14,812	73	14,697	71	387	6,630
Val	2,879	10	3,257	15	86	1,105
Test	2,144	12	1,881	9	50	1,100
Total	19,835	95	19,835	95	523	8,835

3.5 Shared Evaluation Philosophy and Metric Vocabulary

The comparisons in this thesis are designed so that every model within a sub-task is scored by an identical, fixed metric suite, computed by shared evaluation code rather than re-implemented per model. This section defines each metric once. Which metrics apply to which sub-task is itself task-specific, and the actual measured values appear in the relevant results chapters.

3.5.1 Localisation metrics

Localisation accuracy, used for both the ball and the court keypoints, is built on the Euclidean pixel distance between a prediction $\hat{p}_i$ and its ground-truth position p_i,

$$d_i = \|\hat{p}_i - p_i\|_2, \tag{3.1}$$

where both positions are expressed in the original-image pixel space. From this distance two families of metric are derived.

The first is a thresholded accuracy: the fraction of visible targets whose prediction falls within a tolerance τ pixels of the ground truth,

$$\text{acc@}\tau\text{px} = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \mathbb{1}[d_i \leq \tau], \tag{3.2}$$

where $\mathcal{V}$ is the set of ground-truth-visible targets and $\mathbb{1}[\cdot]$ is the indicator function. For ball tracking this quantity is reported at $\tau \in \{5, 10, 20\}$ and written acc@τpx; for court keypoints the identical quantity is the percentage of correct keypoints, written PCK@τpx, and reported at $\tau \in \{5, 7, 10, 25\}$ with $\tau = 7$ used as the primary monitoring threshold during training.

The second family reports the mean error directly. The mean absolute error in pixels, MAE_{px}, is the average of d_i over visible targets. For ball tracking it is additionally broken down per visibility class (classes 1, 2, and 3 of Section 3.1.1; class 0 carries no ball and is excluded), exposing how localisation degrades as the ball becomes harder to see. For court keypoints the mean keypoint error is complemented by the median and maximum keypoint error over detected points, which characterise the typical and worst-case localisation respectively.

3.5.2 Detection metrics

Localisation accuracy presupposes that the target was found; detection metrics measure whether it was found at all. For ball tracking, a binary visibility decision (does the model report a ball, and is there in fact a ball?) yields a precision, recall, and F_1 from the counts of true positives, false positives, and false negatives. A stricter, location-aware variant, following the convention of the TrackNet evaluation [3], counts a prediction as a true positive only when the model reports a ball, the ground truth contains a ball, and the predicted position lies within five pixels of it; a reported ball that is absent in the ground truth or mislocated beyond five pixels is a false positive, and a missed visible ball is a false negative. This produces precision@5px, recall@5px, and F1@5px, which jointly penalise both missed detections and confident mislocalisations.

3.5.3 Event metrics

Bounce detection is scored as an event-timing problem rather than a per-frame classification problem. Because bounce frames make up under 3% of the data (Table 3.2), a per-frame accuracy would be dominated by the negative class and is deliberately not reported. Instead, each model emits a per-frame bounce score; this signal is decoded into discrete event predictions by locating its peaks subject to a minimum spacing between successive events, and the predicted events are matched to ground-truth bounces by greedy one-to-one assignment within a temporal tolerance of $\pm k$ frames. From the resulting true positives, false positives, and false negatives, an event precision, recall, and F_1 at tolerance k are computed via Equation (**??**); the default tolerance is $k = 3$ frames. Two further summaries characterise behaviour beyond a single operating point: the average precision (AP), obtained as the area under the precision-recall curve traced by sweeping the decode threshold, and a tolerance sweep that reports F_1 at $k \in \{0, 1, 2, 3, 5, 7\}$ to show how timing accuracy tightens or relaxes the score. A bounce-versus-hit confusion count, which buckets each predicted bounce according to whether the nearest ground-truth event within the tolerance window is a bounce, a hit, or neither, quantifies the extent to which racket hits are mistaken for bounces. It should be noted that AP and F_1 at a tolerance are different quantities at different operating points and are not interchangeable; a comparison between them is made only at a fixed tolerance, never as a single headline gap.

3.5.4 Geometric and efficiency metrics

For the court sub-task, keypoint accuracy is supplemented by metrics that measure the quality of the downstream geometric registration. A homography is estimated from the predicted keypoints, the visible ground-truth keypoints are projected through it into the canonical court plane, and their displacement from the template is reported in centimetres as a mean and a maximum reprojection error, alongside the rate at which a homography could be estimated at all. The estimation procedure itself, including its robust fitting and the canonical court template, is described where it is used, in Chapter 5.

Finally, efficiency is reported uniformly across all sub-tasks through two quantities: the inference throughput in frames per second (FPS), and the model size as a parameter count in millions. These allow the accuracy of each model to be read against its computational cost, which is key idea to the accuracy-versus- speed trade-offs examined

in the results chapters and to the model-selection rationale of the integrated system (Chapter **??**).

With the datasets, split contracts, and metric definitions now fixed, the following three chapters apply this protocol in turn: ball tracking in Chapter 4, court keypoint detection and homography in Chapter 5, and bounce-event detection in Chapter 6. Each of those chapters also describes the training details specific to its models, including the optimiser, loss function, and hyperparameters.

Chapter 4

Ball Tracking

This chapter applies the protocol of Chapter 3 to the first and most demanding perception sub-task of the system, localising the tennis ball in every frame. The ball is small, often blurred by its own speed, and regularly hidden by players, the net, or the court markings, so its position must be inferred from motion across several frames rather than from any single image. Six models are compared: the five published members of the TrackNet family, V1 through V5, and a single-frame object detector, YOLO11m, included as a representative of the bounding-box paradigm against which the heatmap formulation is contrasted. All six operate on the TrackNet tennis dataset described in Section 3.1 and are scored by the localisation and detection metrics of Sections 3.5.1 and 3.5.2.

The comparison follows a deliberate two-stage selection protocol, which shapes how the results in Sections 4.5 and 4.6 should be read. In the first stage, architecture selection, all six models were trained and evaluated on a smaller subset split in order to compare the architectures cheaply. In the second stage, the single winner of that comparison, TrackNetV4, was retrained on the full dataset split defined in Section 3.4.1. The two stages therefore report different numbers measured on different held-out matches, and a figure from one stage is never placed beside a figure from the other without that distinction being made explicit. This design is a defensible compromise between rigour and cost: it compares six architectures on a common footing, then invests full-dataset training only in the chosen one, and it is the structure through which the chapter answers the first research question (RQ1) on which architecture best localises the ball and what the architectural progression buys in accuracy against speed.

4.1 Task Formulation

Ball tracking is posed here as per-frame localisation rather than as multi-object tracking. For each frame the model must report either a single image coordinate for the ball or an explicit indication that no ball is present, and the per-frame outputs are read as a trajectory without an explicit data-association stage. Two formulations of this task are compared. In the heatmap formulation, which the TrackNet family adopts, the network

predicts a spatial response map whose peak marks the ball centre, and the coordinate is recovered by locating the maximum response; the absence of a ball is expressed naturally as a map with no significant peak [3, 22]. In the bounding-box formulation, which the YOLO detector adopts, the network regresses a box around the ball and its centre is taken as the position, with a confidence score gating whether a ball is reported at all [4]. The contrast between these two formulations on a small, fast, frequently occluded target is the central question of the chapter, and the absence of a ball is represented throughout by the coordinate $(-1, -1)$, the sentinel defined in Section 3.1.

4.2 Preprocessing and Learning Targets

The temporal cue that distinguishes a moving ball from the static background is provided to the models by stacking consecutive frames. For the TrackNet family a sliding window of three consecutive frames is read from a clip and stacked along the channel axis into a nine-channel input, with the ball position of the middle frame as the target; the first and last frames of every clip are skipped so that a complete window of three frames is always available. The frames are resized before stacking, and the resize convention differs by model and by stage. In the architecture-selection comparison of Section 4.5, V1, V4, and V5 operate on a square 256×256 input, V2 and V3 at 512×288, and the YOLO detector at its native 640×640; the final TrackNetV4 of Section 4.6 was retrained at the higher 640×368 resolution, which preserves the broadcast aspect ratio rather than squashing it to a square. These conventions are not intermixed, and every coordinate reported by a model is both expressed and scored in its own resized pixel space rather than being rescaled to a common frame.

The learning target depends on how each model represents the ball. The original V1 design and its V4 descendant do not regress a continuous heatmap but instead predict a quantised intensity class map: a Gaussian centred on the ball, of half-size 20 pixels and variance 10, is rendered and quantised so that each pixel carries an integer intensity class in the range $[0, 255]$, with the peak value 255 at the ball centre, and the network is trained to classify every pixel into one of these 256 classes. V2 and V3 instead regress a continuous Gaussian heatmap with a sigmoid output and a peak value of one, as does V5. The YOLO detector is trained against a fixed bounding box of 20 pixels placed around the labelled point, in the YOLO annotation format. In all cases a frame whose ball is invisible (visibility class 0, or a blank or negative coordinate) produces an all-zero target and contributes the $(-1, -1)$ sentinel rather than a spurious position.

Data augmentation is applied to the training split only, with validation and test frames left unaltered. The augmentation perturbs brightness by up to ± 0.3 of the intensity range and scales contrast by a factor drawn from $[0.7, 1.3]$, and with probability one half it flips the frame window horizontally. The flip is the one augmentation that interacts with the label: when a window is flipped, the ball x-coordinate is remapped to $W - 1 - x$, where W is the frame width, so that the target stays aligned with the image, while an invisible ball (coordinate -1) is left untouched. The same transformation is applied consistently across all three frames of a window so that the temporal cue is preserved.

Table 4.1: The six ball-tracking models compared in this chapter. The TrackNet family shares an encoder-decoder backbone and differs in its output representation and temporal module; YOLO11m is a single-frame bounding-box detector included as the non-heatmap baseline. Parameter counts are computed from the implementations used here; the YOLO11m figure is the published size of the Ultralytics medium model [16].

Model	Output target	Loss	Distinctive module	Params (M)
TrackNet (V1)	256-class intensity map	cross-entropy	VGG encoder-decoder	20.9
TrackNetV2	multi-frame heatmaps	weighted BCE	multiple-in multiple-out	11.3
TrackNetV3	multi-frame heatmaps	weighted BCE + MSE	median background + InpaintNet	11.3
TrackNetV4	256-class intensity map	cross-entropy	motion-attention gating	20.9
TrackNetV5	single heatmap	BCE	temporal self-attention	21.9
YOLO11m	bounding box	detection loss	anchor-free box regression	20.1

4.3 Models

The five TrackNet variants share a common lineage, an encoder-decoder convolutional network with skip connections that ingests the nine-channel frame stack and produces a spatial output at the input resolution [3]. They differ in the output representation, in the depth of the encoder-decoder, and in the temporal-reasoning module each one adds. Table 4.1 summarises the six models along the axes that matter for the comparison, and the architecture of each of the five TrackNet variants is drawn in Figure 4.1, panels (a) to (e).

TrackNet (V1), in panel (a), follows the original design [3]: a four-stage VGG-style encoder with max-pooling, a convolutional bottleneck, and a symmetric decoder whose transposed convolutions are concatenated with the matching encoder features through skip connections. A final 1×1 convolution maps the decoded features to 256 channels, one per intensity class, and the position is recovered by taking the per-pixel argmax to form an intensity image and locating its global peak; a peak below an intensity threshold yields the no-ball sentinel. This is a multiple-in single-out design, three frames in and one ball position out.

TrackNetV2, in panel (b), reorganises the backbone into a multiple-in multiple-out form that predicts a sigmoid Gaussian heatmap for every frame in the window in a single pass, reducing redundant computation [25]. Its encoder-decoder is shallower than V1's, three stages rather than four, with nearest-neighbour upsampling in the decoder, which roughly halves the parameter count to 11.3 million. The coordinate is the argmax of the relevant frame's heatmap when its peak exceeds 0.5, and the sentinel otherwise.

TrackNetV3, in panel (c), builds on the V2 backbone with two additions aimed at robustness [26]. A per-pixel median-background channel, computed across the input window, is concatenated to the frame stack so that the network sees an explicit estimate of the static scene. Separately, an inpainting network, a small one-dimensional U-Net that

takes a sequence of predicted coordinates with a visibility flag and outputs a rectified sequence, repairs missed detections by reasoning over the predicted path; it is trained with a mean-squared-error objective on the trajectory and operates as a post-processing stage over the tracker's per-frame peaks.

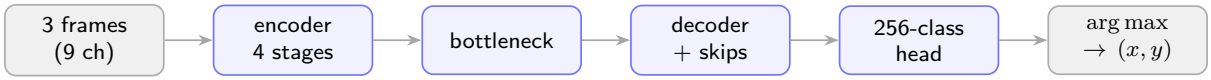
TrackNetV4, in panel (d), is the model selected in Section 4.5. It keeps the V1 backbone and 256-class output, and adds a lightweight motion-attention module [27]. The module forms absolute differences between successive input frames, passes the concatenated difference maps through two convolutions and a sigmoid to produce a single-channel spatial attention map, and multiplies this map into the decoder features at two resolutions. The effect is to amplify regions of recent motion, where a fast ball is likely to be, while suppressing the static background, at a negligible parameter cost over V1 (20.9 million for both).

TrackNetV5, in panel (e), encodes each of the three frames independently with a shared VGG-style encoder, then fuses their deepest features with a multi-head temporal self-attention layer that treats the three frames as a short sequence at each spatial position, before decoding from the attended middle-frame features to a single sigmoid heatmap. It is the largest of the family at 21.9 million parameters. As noted in Section 2.2.2, V5 is a preprint variant [28] and is included to test whether learned temporal attention improves on the explicit motion cue of V4.

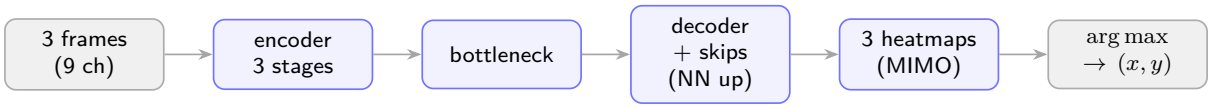
YOLO11m is a single-frame, anchor-free object detector from the Ultralytics YOLO family [4, 16]. Unlike the TrackNet models it sees one frame at a time and regresses a bounding box whose centre is taken as the ball position, with a confidence threshold of 0.3 below which no ball is reported. It carries no temporal cue and serves as the bounding-box baseline against which the heatmap formulation is measured.

4.4 Training Details

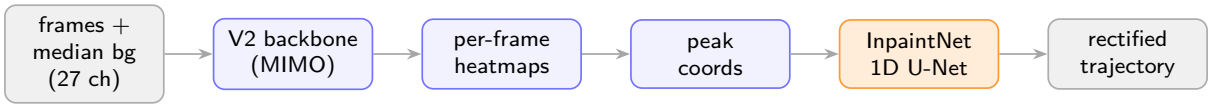
All models were trained on a single GPU with a learning-rate schedule that reduced the rate by half when the validation loss plateaued, and with early stopping that halted training when the validation accuracy at five pixels failed to improve for a fixed number of epochs. Within this common regime the models differ in optimiser, loss, and the hyperparameters listed in Table 4.2. The two 256-class models, V1 and V4, were trained with the Adadelta optimiser and a cross-entropy loss over the intensity classes; the heatmap-regressing V2 and V3 trackers used the Adam optimiser [47] with a weighted binary cross-entropy that emphasises the sparse positive pixels in the manner of focal loss [18]; V5 used Adam with a plain binary cross-entropy on its sigmoid heatmap; and the InpaintNet rectifier of V3 was trained separately with mean-squared error on the coordinate sequence. YOLO11m was trained through the Ultralytics framework with its own composite detection loss and optimiser defaults. The differences in batch size and epoch budget in Table 4.2 reflect the differing memory footprints and convergence behaviour of the models rather than a tuned search, and every model was trained under the identical subset split of Section 4.5 so that the comparison is like-for-like.



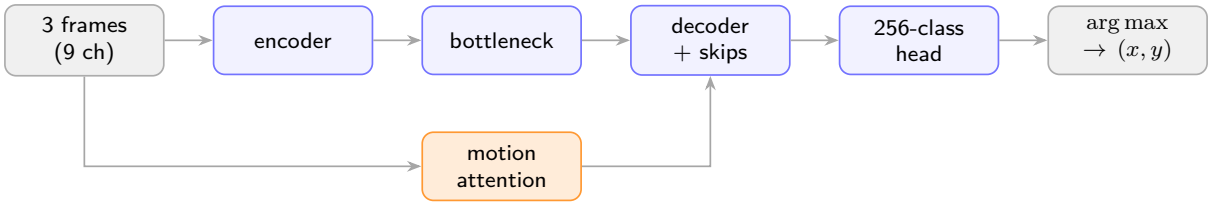
(a) TrackNet (V1): four-stage VGG encoder-decoder with skip connections and a 256-class intensity head; multiple-in, single-out.



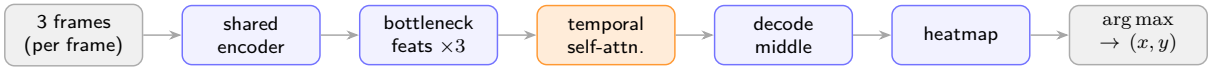
(b) TrackNetV2: shallower three-stage backbone that emits one heatmap per input frame in a single multiple-in multiple-out pass.



(c) TrackNetV3: the V2 tracker fed an extra median-background channel, followed by an InpaintNet that rectifies the trajectory (shaded).



(d) TrackNetV4: the V1 backbone with a motion-attention branch (shaded) that gates the decoder towards moving regions; the selected model.



(e) TrackNetV5: each frame is encoded independently and the three bottleneck features are fused by temporal self-attention (shaded) before decoding the middle frame.

Figure 4.1: Architecture of the five TrackNet-family variants compared in this chapter. All share an encoder-decoder backbone that ingests stacked frames and produces a spatial output decoded to a ball coordinate; they differ in the output representation and in the temporal module each one adds, shaded for the trajectory rectifier of V3, the motion attention of V4, and the temporal self-attention of V5.

Table 4.2: Training configuration of the six models. All used a reduce-on-plateau learning-rate schedule and early stopping; they differ in optimiser, loss, and budget. The two 256-class models (V1, V4) use Adadelta with cross-entropy; the heatmap models use Adam.

Model	Optimiser	Loss	Init. LR	Batch	Epochs
TrackNet (V1)	Adadelta	cross-entropy	1.0	4	50
TrackNetV2	Adam	weighted BCE	10^{-3}	10	30
TrackNetV3	Adam	weighted BCE / MSE	10^{-3}	10	30
TrackNetV4	Adadelta	cross-entropy	1.0	4	50
TrackNetV5	Adam	BCE	5×10^{-5}	8	60
YOLO11m	SGD	detection loss	10^{-4}	16	50

4.5 Model Selection: the Six Model Subset Comparison

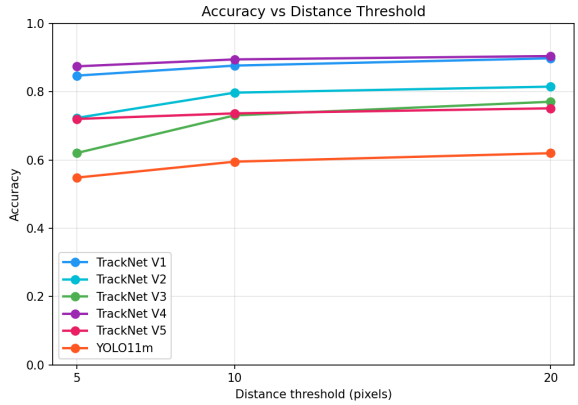
The first stage of the protocol compared all six models on a subset split small enough to train every architecture at modest cost. Matches game1 and game2 were partitioned by clip into 70% training, 15% validation, and 15% test fractions, and game7 was held out in its entirety as an unseen-match test set on which the comparison metrics were measured. This subset split is distinct from the full split used for the final model in Section 4.6: it trains on far fewer matches and reports on a different held-out match (game7 rather than game10), so its numbers must be read as an internally consistent ranking of architectures, not as the system's final accuracy. The results are given in Table 4.3.

Tracking consistency (TC), reported here for the first time, is the mean frame-to-frame displacement of the predicted ball position between consecutive frames; a lower value indicates a smoother, less jittery track, which matters because the downstream bounce detector and court projection consume the trajectory rather than isolated points. The accuracy curves over the tolerance sweep, together with bar charts of the error, F_1, and throughput metrics and the per-visibility error breakdown, are shown in Figure 4.2.

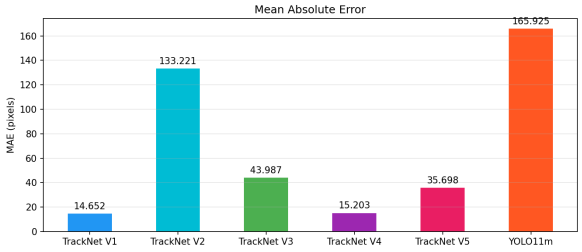
4.5.1 The case for TrackNetV4

TrackNetV4 was selected on three grounds. First, it is the strongest localiser at strict tolerance: it attains the highest accuracy at all three thresholds, and in particular the highest acc@5px (0.875), the most demanding and the most decision-relevant of the localisation metrics for a ball whose position must later feed a geometric line call. Second, it produces the smoothest track, with the lowest tracking consistency of all six models (10.44 pixels), below V1's 13.98 and far below the V2, V3, and YOLO values; a steady trajectory directly benefits the downstream bounce detector and the court projection. Third, it is among the fastest, at 53.07 frames per second, effectively tied with the quickest model and between two and four times the throughput of V2, V3, and V5. TrackNetV4 therefore sits at the favourable corner of the accuracy-speed trade-off, offering the best strict-tolerance accuracy at a negligible cost in speed.

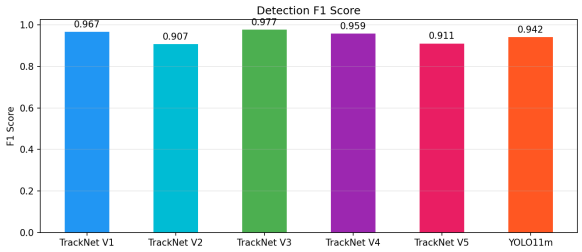
The remaining models were not selected for reasons that the table makes concrete. V1 is the only genuine rival: it records a marginally lower mean error (14.65 against 15.20 pixels), higher recall (0.966 against 0.937), and a few more frames per second, but



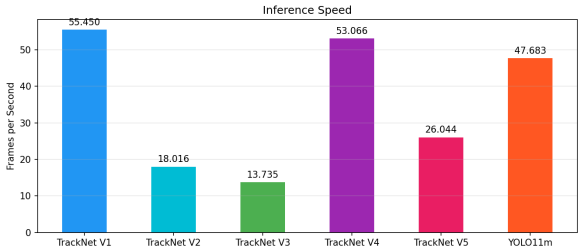
(a) Accuracy versus distance tolerance.



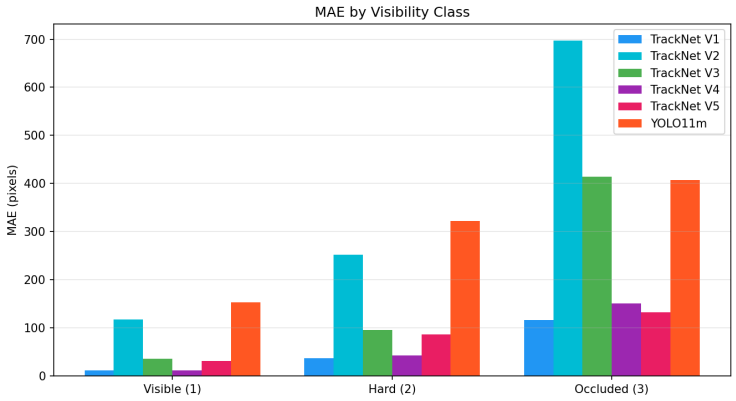
(b) Mean absolute error.



(c) Detection F_1.



(d) Throughput in frames per second.



(e) Mean absolute error by visibility class.

Figure 4.2: Six-model comparison on the subset test match (game7); the plotted values are those tabulated in Table 4.3. (a) Localisation accuracy across the $\{5, 10, 20\}$-pixel tolerance sweep, where TrackNetV4 and V1 lead while YOLO11m trails. (b) Mean absolute error, (c) detection F_1, and (d) inference throughput per model. (e) Mean absolute error split by visibility class, showing that all models degrade from clearly visible to occluded balls, with the heaviest tails for V2 and YOLO11m. All throughput values were measured on one GPU and are comparable only within this figure (Section 4.6).

Table 4.3: Six-model comparison on the subset test match (game7). All figures are measured on the same held-out match under one protocol; in particular every frames-per-second figure in this table was timed on the same GPU and is internally comparable, but is *not* comparable with the final-model throughput of Table 4.4, which was measured on different hardware (Section 4.6). Bold marks the best value in each column; lower is better for MAE_{px} and tracking consistency (TC). The detection columns Prec, Rec, and F_1 are the visibility-based detection metrics of Section 3.5.2, scoring only whether a ball is reported when one is in fact present; they are therefore distinct from the localisation accuracy acc@τpx and from the stricter location-aware precision@5px, recall@5px, and F_1@5px, which additionally require the prediction to fall within five pixels and are reported for the final model in Table 4.4.

	Localisation				Detection (visibility)				
Model	acc@ 5px	acc@ 10px	acc@ 20px	MAE_{px}	Prec	Rec	F_1	TC	FPS
TrackNet (V1)	0.847	0.877	0.898	**14.65**	0.968	0.966	0.967	13.98	**55.45**
TrackNetV2	0.723	0.797	0.815	133.22	**0.998**	0.831	0.907	25.99	18.02
TrackNetV3	0.620	0.731	0.771	43.99	0.958	**0.997**	**0.977**	34.85	13.73
TrackNetV4	**0.875**	**0.895**	**0.905**	15.20	0.982	0.937	0.959	**10.44**	53.07
TrackNetV5	0.720	0.737	0.752	35.70	0.978	0.852	0.911	13.31	26.04
YOLO11m	0.548	0.595	0.620	165.93	0.954	0.930	0.942	74.79	47.68

its acc@5px is lower and its track is noticeably jitterier, so V4's motion attention is read as buying sharper and steadier localisation at the strict tolerance for almost no speed penalty. V2 attains the highest precision but its recall collapses to 0.831 and its mean error is catastrophic at 133 pixels, indicating that it frequently fails to localise the ball at all on this subset. V3 has the best recall and F_1, meaning it finds the ball most often, yet it records the lowest acc@5px of the TrackNet family (0.620) and the lowest throughput (13.73 frames per second): its inpainting rectifier recovers the presence of the ball without recovering its precise position. V5 is middling on every axis, suggesting that learned temporal self-attention adds parameters and cost without an accuracy gain over the explicit motion cue of V4 on this data. YOLO11m is the weakest localiser overall, with the lowest acc@5px (0.548), a mean error of 166 pixels, and by far the jitteriest track (74.79), which is consistent with the expectation that box-centre regression localises a tiny ball less precisely than dense heatmap regression.

These observations answer RQ1 directly. The progression from V1 to V5 is not monotone in accuracy: the later variants are not uniformly better, and the strongest strict-tolerance localiser is V4, whose motion-attention mechanism proves to be the most effective of the temporal modules tried. The single-frame bounding-box detector is clearly inferior to the heatmap trackers for this target, which supports the thesis's broader claim that heatmap regression is the more suitable formulation for a small, fast ball.

4.6 Final Model's Results: TrackNetV4 on the Full Dataset

The second stage retrained the selected model, TrackNetV4, on the full dataset split of Section 3.4.1: matches game1 through game9 for training and validation, and game10

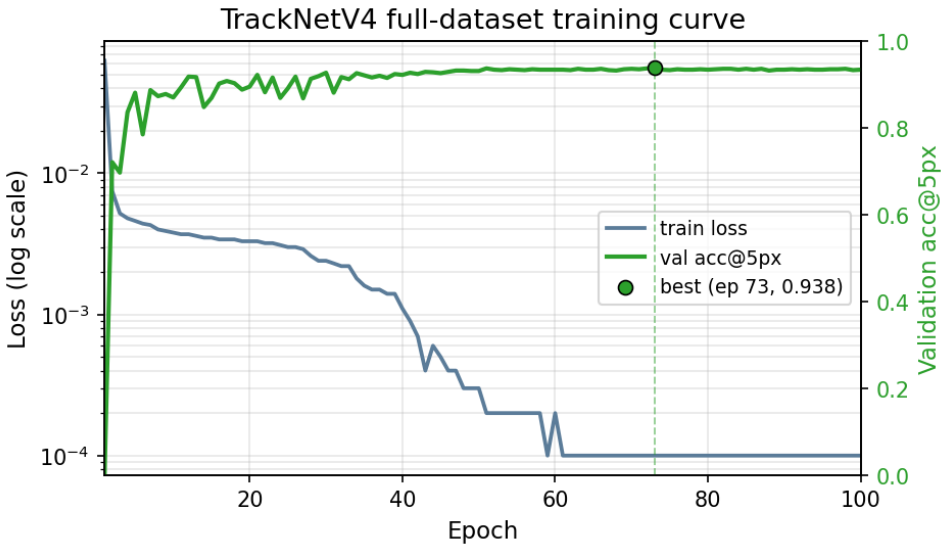


Figure 4.3: Training curve of the final TrackNetV4 on the full dataset split (game1–game9 for training and validation). The training loss is plotted on a logarithmic scale (left axis) and validation accuracy at five pixels on a linear scale (right axis), against the epoch count. The training loss falls by roughly three orders of magnitude; validation acc@5px climbs to a plateau around 0.93 and peaks at 0.938 at epoch 73 (marked), the checkpoint carried forward for evaluation.

held out in its entirety as the test match. Its closest rival from the selection stage, the original TrackNet (V1), was retrained on the identical split as a reference baseline, to test whether the architecture ranking of Section 4.5 survives the move to the full dataset. Training of TrackNetV4 ran for 100 epochs and recorded a best validation acc@5px of 0.938 at epoch 73; its loss and validation-accuracy curves are shown in Figure 4.3. Both models were then evaluated on the unseen game10 test match, and the resulting test metrics are reported in Table 4.4. These figures are the headline ball result of the thesis; they are measured on a different and larger split than the selection numbers of Table 4.3 and must not be compared column-for-column with them.

Several features of Table 4.4 deserve comment, and they are relevant to how the numbers are quoted elsewhere in the thesis. First, the selection-stage ranking holds on the full dataset: TrackNetV4 again leads its closest rival V1 on strict-tolerance accuracy (acc@5px of 0.956 against 0.948) and on mean error in every visibility class, while V1 retains the marginally higher precision (1.000 against 0.996), the smoother track (a tracking consistency of 7.85 against 8.82 pixels), and a few more frames per second (43.65 against 40.82). This is the same accuracy-against-smoothness trade-off seen at selection, now confirmed on a larger and previously unseen match, and it justifies carrying V4 forward as the system's ball tracker. Second, the strict-tolerance accuracy of V4 rises substantially relative to the selection stage, from 0.875 on the subset to 0.956 on the full split, and it does so even though the final model is scored on a finer 640×368 grid, on which five pixels is a stricter tolerance than on the selection stage's 256×256 grid; the larger and more varied training set, together with the higher resolution, sharpens localisation on clearly visible balls. The two stages' mean errors are measured on these different grids and so are not directly comparable in raw pixels, but the final aggregate of 12.13 pixels for V4 is dominated, as at selection, by a heavy tail of difficult cases: on

Table 4.4: Final metrics of TrackNetV4 and its closest rival, the original TrackNet (V1), both retrained on the full dataset split and evaluated on the held-out test match (game10) at the best validation checkpoint. Bold marks the better of the two values in each column; lower is better for the MAE columns and tracking consistency (TC). The four MAE columns give the mean localisation error in pixels: overall (all) and, split by visibility class, for clearly visible (vis1), hard-to-identify (vis2), and occluded (vis3) balls. P@5px, R@5px, and F_1@5px are the location-aware detection metrics of Section 3.5.2. The frames-per-second figures were measured on different hardware and under a different timing protocol than Table 4.3 and are not comparable with that table (see text).

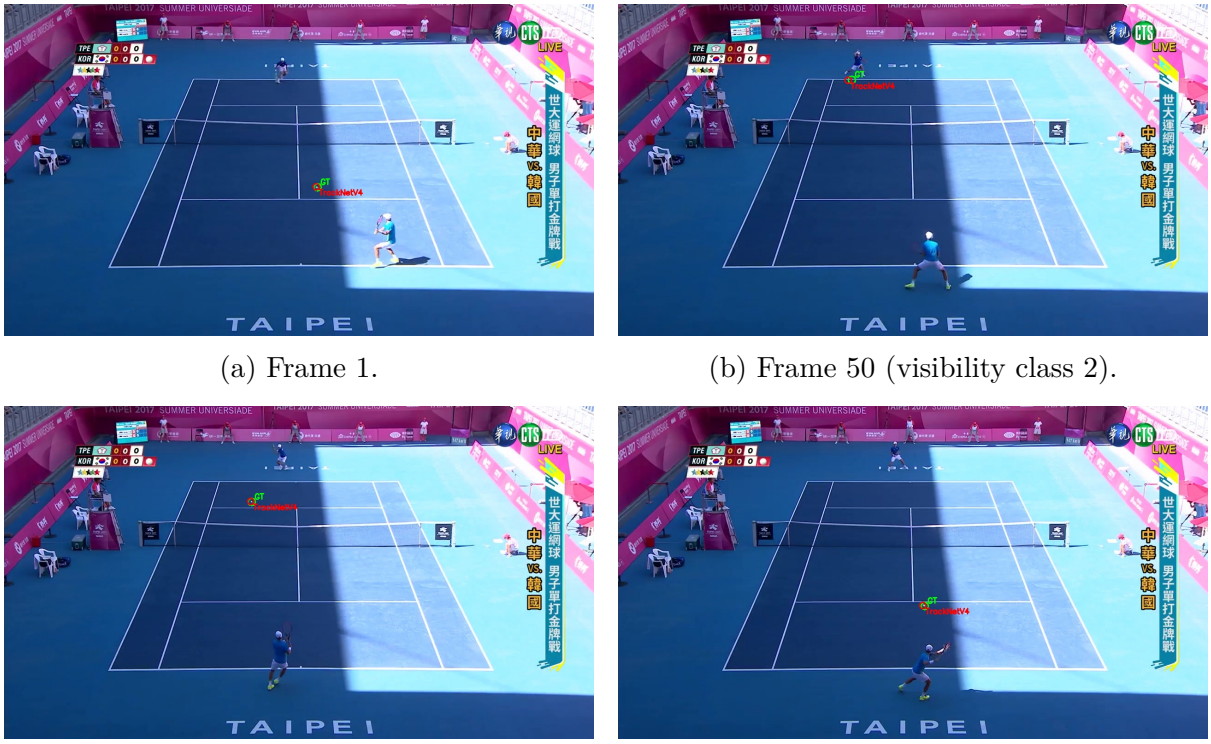
	Localisation acc			MAE (px)				Detection			Detection @5px				
Model	acc@ 5px	acc@ 10px	acc@ 20px	all	vis1	vis2	vis3	Prec	Rec	F_1	P@ 5px	R@ 5px	F_1@ 5px	TC	FPS
TrackNet (V1)	0.948	0.952	0.954	16.08	9.81	78.28	317.72	**1.000**	0.957	0.978	**0.990**	0.948	**0.969**	**7.85**	**43.65**
TrackNetV4	**0.956**	**0.959**	**0.963**	**12.13**	**6.89**	**63.87**	317.72	0.996	**0.972**	**0.984**	0.980	**0.956**	0.968	8.82	40.82

clearly visible balls (visibility class 1) the mean error is tight at 6.89 pixels, whereas on the rare hard-to-identify balls (class 2) it is 63.87 pixels and on the still rarer occluded balls (class 3) it is 317.72 pixels. For this reason acc@5px, and not the mean error, is quoted as the headline localisation figure throughout. The track remains smooth, with a tracking consistency of 8.82 pixels, and precision is near-perfect at 0.996 on game10, so that almost every reported ball is a true positive, while the location-aware F_1@5px of 0.968 confirms that the great majority of detections are also well localised.

4.7 Analysis

The selection result and the final metrics together support a coherent picture of why TrackNetV4 succeeds and where it remains limited. Its advantage traces to the motion-attention module: by gating the decoder with a map of inter-frame motion, the network concentrates its capacity on the regions where a fast ball actually appears, which sharpens the response peak and, as the tracking consistency shows, steadies the trajectory between frames. This is achieved at essentially no parameter cost over the V1 backbone, which is why V4 dominates V1 on strict-tolerance accuracy and smoothness while matching it on speed. The comparison with V5 is instructive in the same vein: the explicit, almost free motion cue of V4 outperformed the heavier learned temporal self-attention of V5 on this data, indicating that for this target a strong inductive bias towards motion is more effective than a general attention mechanism left to learn the same cue.

The per-visibility breakdown locates the residual difficulty precisely. The model is accurate when the ball is clearly visible, which is the common case, but its error grows by an order of magnitude on the motion-blurred and occluded frames of visibility classes 2 and 3. Figure 4.4 shows the final model's output on four frames sampled across a held-out game10 rally, where the predicted ball position (red) tracks the ground truth (green) to within a few pixels throughout. The dominant failure modes are the ones that the task formulation anticipates: a ball occluded by a player or the net, which leaves no peak for the heatmap to find; a ball moving fast enough to blur across many pixels, which broadens and weakens the response; and a ball near or beyond the frame edge, which has no well-defined centre. These cases are rare in the dataset and are reflected in the heavy tail of the mean error rather than in the strict-tolerance accuracy, but they are the



(a) Frame 1.

(b) Frame 50 (visibility class 2).

(c) Frame 100.

(d) Frame 150.

Figure 4.4: Qualitative output of the final TrackNetV4 model on four frames sampled across a held-out game10 rally. In each frame the ground-truth ball position (green, GT) and the model's prediction (red, TrackNetV4) are overlaid; the two markers coincide to within a few pixels throughout, including the motion-blurred mid-court frame of visibility class 2 (b). The ball is localised accurately across the full range of court positions and play phases seen in the rally.

natural targets for the future work discussed in Chapter **??**, and they are precisely the situations in which the downstream trajectory rectification and event detection must be robust.

In summary, the two-stage comparison answers RQ1 by identifying TrackNetV4 as the best ball-tracking model among the six considered, on the grounds of strict-tolerance accuracy and trajectory smoothness at competitive speed, and by showing that the architectural progression through the family is not monotone and that the heatmap formulation is clearly preferable to single-frame bounding-box detection for this target. The selected model carries forward to the integrated system of Chapter **??** as the ball-tracking component, where its smooth, well-localised trajectory feeds the bounce detector of Chapter 6

and the court projection of Chapter 5.

Chapter 5

Court Keypoint Detection and Homography

This chapter applies the protocol of Chapter 3 to the second perception sub-task, locating the fixed landmarks of the tennis court in a single broadcast frame and using them to register the image to a court-plane coordinate system. Where ball tracking must follow a small moving target through time, court detection faces the opposite problem: the landmarks are static and well defined, but they must be located precisely, because their pixel coordinates feed a homography whose accuracy in court metres degrades quickly with keypoint error. Four heatmap models are compared on the court-keypoint dataset of Section 3.2, a from-scratch baseline and three models built on pretrained backbones of very different sizes, and each is scored both by the keypoint-localisation metrics of Section 3.5.1 and by the geometric registration metrics of Section 3.5.4. The chapter answers the second research question (RQ2) on which backbone gives the best accuracy against latency trade-off for fourteen-keypoint court detection, and whether the chosen model is reliable enough to drive a homography into court-metric space.

Unlike the ball comparison of Chapter 4, the court comparison is a single-stage one: all four models were trained and evaluated on the one court split of Section 3.4.2, with no separate subset-selection stage, so the numbers in this chapter are directly comparable across models. The model carried forward into the integrated system of Chapter **??** is selected here on the combined evidence of accuracy, geometric registration quality, throughput, and size.

5.1 Task and Data Specifics

Court detection is posed as the localisation of fourteen fixed court landmarks in a single RGB frame, each landmark predicted as the peak of a dedicated heatmap channel. The fourteen landmarks follow the ordering of the TennisCourtDetector collection [46] that the dataset is drawn from, and that ordering is fixed throughout the data pipeline, the models, and the geometric template, so it is documented once here. Table 5.1 lists the fourteen landmarks together with their coordinates in the canonical court template described in Section 5.4, and Figure 5.1 draws them on a scale court diagram. The four outer corners (indices 0 to 3) are the doubles-court corners; indices 4 to 7 are the points where the singles sidelines meet the two baselines; indices 8 to 11 are the points where the singles sidelines meet the two service lines; and indices 12 and 13 are the centre points of the two service lines, where the centre service line meets them.

Table 5.1: The fourteen annotated court keypoints in the TennisCourtDetector ordering used throughout the court pipeline, with their coordinates in the canonical court template of Section 5.4. The template origin is the court centre, the x axis runs along the court width towards the right sideline, and the y axis runs along the court length towards the top baseline; all coordinates are in metres. A fifteenth heatmap channel (the court centre) is constructed rather than annotated, as described in the text.

Index	Landmark	x (m)	y (m)
0	top-left outer (doubles) corner	−5.485	+11.885
1	top-right outer (doubles) corner	+5.485	+11.885
2	bottom-left outer (doubles) corner	−5.485	−11.885
3	bottom-right outer (doubles) corner	+5.485	−11.885
4	top-left singles baseline corner	−4.115	+11.885
5	bottom-left singles baseline corner	−4.115	−11.885
6	top-right singles baseline corner	+4.115	+11.885
7	bottom-right singles baseline corner	+4.115	−11.885
8	top-left service-line point	−4.115	+6.400
9	top-right service-line point	+4.115	+6.400
10	bottom-left service-line point	−4.115	−6.400
11	bottom-right service-line point	+4.115	−6.400
12	top centre service point	0.000	+6.400
13	bottom centre service point	0.000	−6.400

The dataset is the publicly distributed TennisCourtDetector collection introduced in Section 3.2, and Figure 3.3 there shows two annotated frames. Its split is the deterministic, seedless partition of Section 3.4.2: the 6 630 training records are used as supplied, and the 2 211 validation records are sorted by identifier and divided into 1 105 validation records and the remainder for test.

5.2 Preprocessing and Learning Targets

Every frame is resized to a fixed 360×640 input (height by width), and the predicted keypoints are scaled back to the original frame resolution for evaluation. The original frames of this dataset are 1280×720, so the rescaling factor from the input back to the original is uniform at a factor of two on each axis; this factor matters for the strict-tolerance results and is taken up again in Section 5.6. Pixel intensities are always divided by 255 to lie in $[0, 1]$, and for the three models built on ImageNet-pretrained backbones the per-channel ImageNet mean and standard deviation are additionally subtracted and divided, so that the inputs match the statistics the backbones were pretrained on; the from-scratch baseline uses the plain $[0, 1]$ scaling only.

The learning target for each annotated keypoint is a CenterNet-style Gaussian blob centred on the landmark [22], rendered into that keypoint's heatmap channel at the model's output resolution. The Gaussian has a fixed radius in heatmap pixels, with a standard deviation set to one sixth of the blob diameter, and overlapping blobs are combined by taking the per-pixel maximum. Because the models emit heatmaps at different output strides (Section 5.3), the radius is set per model so that the relative spatial spread of the target is comparable: it is fifteen pixels for the two models that

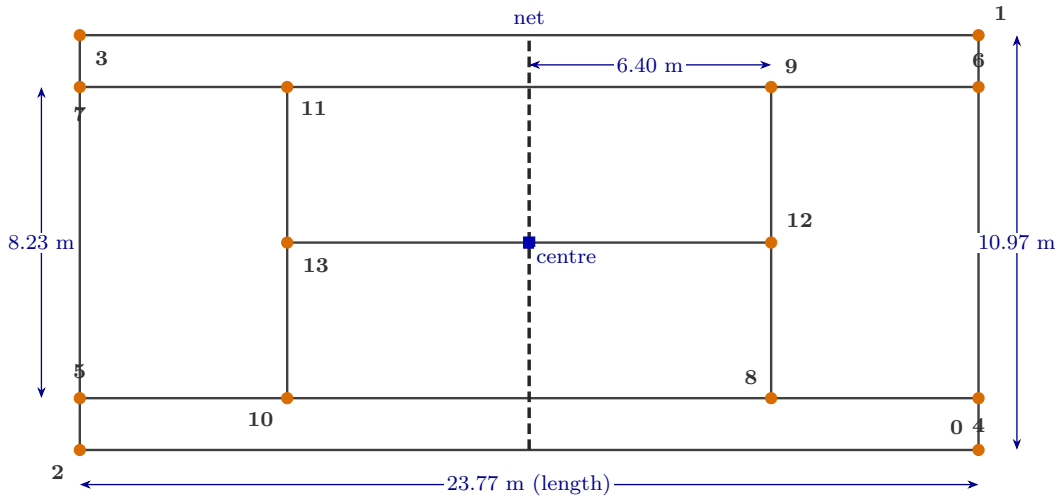


Figure 5.1: The canonical tennis court template and the fourteen annotated keypoints, drawn to scale with the court length along the horizontal axis for compactness. Orange discs mark the fourteen annotated landmarks of Table 5.1, numbered by their channel index; the blue square at the centre marks the constructed fifteenth channel. The dashed line is the net. Dimensions are the International Tennis Federation court specification: a doubles court of 23.77×10.97 m, a singles width of 8.23 m, and service lines 6.40 m from the net.

output at the input resolution and eight pixels for the two that output at one quarter of it. A keypoint that falls outside the frame, or is otherwise marked invalid, contributes no blob to its channel and the corresponding coordinate carries the $(-1, -1)$ sentinel of Section 3.1 rather than a spurious position.

Data augmentation is applied to the training split only. It perturbs brightness by up to ± 0.3 of the intensity range, scales contrast by a factor drawn from $[0.7, 1.3]$, and with probability one half flips the frame horizontally. As in the ball pipeline, the horizontal flip is the augmentation that interacts with the labels, and it does so more intricately here than for a single ball: flipping the image left to right not only remaps each keypoint x-coordinate to $W - 1 - x$ but also permutes the keypoint indices, because a left landmark becomes a right landmark. The permutation swaps the symmetric left and right pairs (the two outer corners, the two singles baseline corners, the singles points, and the service points) while leaving the two centre service points unchanged, and it is applied as a fixed index map so that each channel continues to predict the correct landmark after the flip.

5.3 Models

Four models are compared, chosen to span the design space from a faithful reimplementation of the dataset's own baseline to backbones of widely differing capacity and cost. All four take a single RGB frame and emit a fifteen-channel heatmap, and all four are trained under the identical regime described below, so that the comparison isolates the effect of the backbone and the output resolution. Table 5.2 summarises them along the axes that shape the results.

TrackNet-court is a faithful reimplementation of the baseline shipped with the dataset [46], the same VGG-style encoder-decoder backbone used for ball tracking in Chapter 4

Table 5.2: The four court-keypoint models. All emit a fifteen-channel heatmap (fourteen landmarks plus the constructed centre). Output stride is the ratio of the input resolution to the heatmap resolution: the two stride-1 models predict at the full 360×640 input grid, the two stride-4 models at a 90×160 grid. Parameter counts are computed from the implementations used here.

Model	Backbone	Decoder / head	Out. stride	Gauss. radius	ImageNet norm.	Params (M)
TrackNet-court	VGG-style encoder-decoder(from scratch)	transposed-conv decoder with skips	1	15	no	20.8
ResNet50	ResNet-50(ImageNet)	three transposed-conv blocks	4	8	yes	34.0
HRNet	HRNet-W32(timm features)	stride-4 feature tap + 1×1	4	8	yes	30.9
MobileNetV3	MobileNetV3-Small(ImageNet)	U-Net decoder	1	15	yes	1.28

[3], adapted to take a single three-channel frame rather than a stacked window and to emit fifteen channels rather than one. Its decoder upsamples back to the full input resolution through transposed convolutions with skip connections, so it predicts at output stride one, a 360×640 heatmap per channel, and it is the only model trained from scratch without ImageNet normalisation.

ResNet50 pairs an ImageNet-pretrained ResNet-50 backbone [6] with the simple-baseline pose decoder of Xiao et al. [31]: three transposed-convolution blocks that upsample the stride-32 backbone features to a stride-4 heatmap, followed by a 1×1 convolution to fifteen channels. It predicts at output stride four, a 90×160 heatmap, and it is the heaviest model at thirty-four million parameters.

HRNet is built on an HRNet-W32 backbone obtained through the `timm` library [7]. It should be noted that the implementation taps a stride-4 feature map from the backbone's feature pyramid and applies a 1×1 convolutional head, rather than using HRNet's native full-resolution branch; it is therefore a stride-4 model in the same sense as ResNet50, and this design choice bears directly on its strict-tolerance accuracy, as Section 5.6 discusses. It carries thirty-one million parameters.

MobileNetV3 pairs an ImageNet-pretrained MobileNetV3-Small backbone [8], the efficient inverted-residual architecture in the MobileNet line [32], with a U-Net-style decoder that upsamples all the way back to the full input resolution. It therefore predicts at output stride one, a 360×640 heatmap, like TrackNet-court, but at a fraction of the size: at 1.28 million parameters it is roughly one sixteenth the size of TrackNet-court and one twenty-fifth the size of ResNet50.

All four models output raw logits, with the sigmoid applied inside the loss and at evaluation. They were trained with a common recipe, the Adam optimiser [47] with a CenterNet-style penalty-reduced focal loss on the sigmoid of the predicted heatmaps [22], which drives the response towards one at each keypoint peak while reducing the penalty on the pixels around it in proportion to their Gaussian target value, a learning rate of 10^{-4} halved on plateau, a batch size of eight, and early stopping with a patience of thirty epochs. The checkpoint kept for each model is the one with the best validation accuracy at the seven-pixel tolerance, measured in the resized input space; the distinction between

that monitoring metric and the test metric reported below is discussed in Section 5.6. A keypoint coordinate is decoded from a predicted heatmap by locating the peak channel response, refining it to sub-pixel precision with a local three-by-three intensity-weighted mean, and scaling by the output stride and the input-to-original factor; a peak below a confidence threshold of 0.3 yields the no-keypoint sentinel.

5.4 Homography Estimation

The motivation for detecting court keypoints precisely is that they anchor a planar homography, the map from image pixels (u, v) to court-plane coordinates (X, Y) in metres that turns a detected ball position into a measurement on the court and so makes a downstream in-or-out decision possible. The court is planar and the camera projection is perspective, so the image-to-court map is a single 3×3 homography, the natural object to estimate from corresponding points [33]. The canonical court template against which the homography is fitted is the metric layout of Table 5.1 and Figure 5.1: a doubles court of 23.77×10.97 m with the origin at the court centre, built directly from the International Tennis Federation court dimensions, the singles width of 8.23 m and the service lines 6.40 m from the net.

The homography is estimated from the detected keypoints with a random-sample-consensus fit [5], which is essential here because individual keypoints can be missed or mislocated and a least-squares fit to all of them would be corrupted by a single gross outlier. The procedure filters out any keypoint carrying the missing sentinel, then fits a homography from the canonical template points to their detected image positions with a RANSAC reprojection threshold of five pixels, which keeps the threshold in the intuitive image-pixel units. At least four visible keypoints are required for a homography to be defined, and the fit is accepted only if at least six of the points are RANSAC inliers, so that an estimate is reported only when it is well supported; the accepted image-to-court map is then the inverse of the fitted template-to-image transform. The routine returns a three-part result, the image-to-court homography (or an explicit absence when the fit fails), the inlier mask over the fourteen keypoints, and the mean reprojection error over the inliers in pixels, so that a caller can both use the map and judge its quality.

Two evaluation quantities follow from this estimate, as defined in Section 3.5.4. The homography success rate is the fraction of test frames for which an accepted homography could be estimated at all, and it measures how often the detector produces a usable geometric registration. The mean reprojection error in centimetres measures the geometric quality of an accepted homography: the visible ground-truth keypoints are projected through the predicted homography into the court plane and their displacement from the canonical template positions is measured in court metres, then expressed in centimetres. This figure is more interpretable than a pixel error, since it is stated in the physical units in which an in-or-out margin is judged, although it should be read with the caveat noted in Section 5.6 that it combines keypoint-localisation error with homography-fit error in a single number.

5.5 Results

The four models were evaluated on the 1 100-record court test set, with all keypoint coordinates scaled back to the original 1280×720 resolution before scoring. Table 5.3

Table 5.3: Court-keypoint results on the held-out test set, with predictions in original 1280×720 pixel space. PCK@τpx is the percentage of correct keypoints within τ pixels; MAE_{px} is the mean keypoint error; reproj. is the mean court-plane reprojection error in centimetres; succ. is the homography success rate. Bold marks the best value in each column; lower is better for MAE_{px} and the reprojection error. The frames-per-second figures were all measured in a single comparison run on one machine and are internally comparable as a ranking, but they include the per-frame keypoint decoding and homography estimation and are not a pure inference-throughput claim.

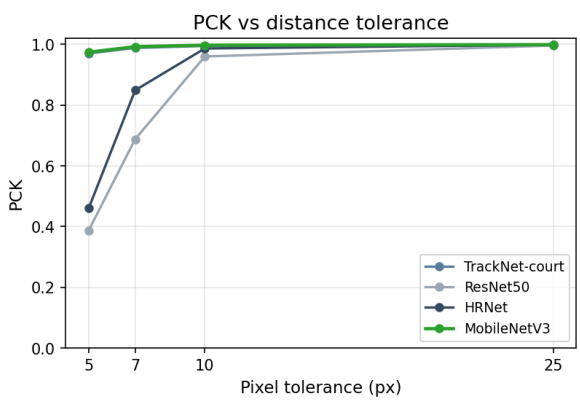
	PCK (localisation)					Homog.		
Model	@5px	@7px	@10px	@25px	MAE_{px}	reproj. (cm)	Params (M)	FPS
TrackNet-court	0.970	0.988	0.995	0.998	3.46	**7.47**	20.8	4.45
ResNet50	0.388	0.688	0.960	0.996	8.24	30.58	34.0	8.67
HRNet	0.460	0.849	0.986	0.998	5.98	30.78	30.9	10.42
MobileNetV3	**0.974**	**0.992**	**0.997**	**0.999**	**2.46**	7.67	**1.28**	**21.00**

reports the keypoint-localisation metrics, the geometric registration metrics, and the size and throughput of each model; Figure 5.2 plots the accuracy, error, and size relationships, and Figure 5.3 breaks the seven-pixel accuracy down by keypoint.

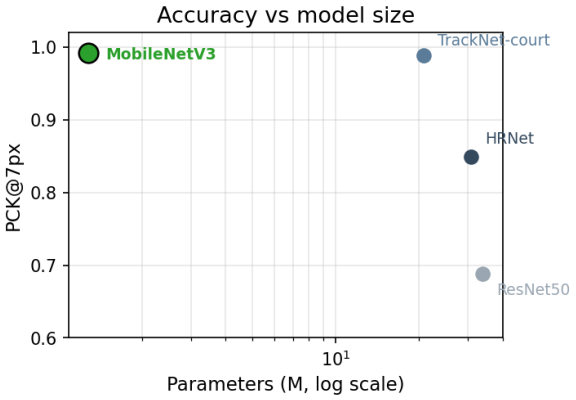
The headline result is that MobileNetV3, by far the smallest of the four models, is also the most accurate on every metric but one. It attains the best accuracy at all four tolerances, including the most demanding five-pixel and seven-pixel thresholds (0.974 and 0.992), the lowest mean keypoint error (2.46 pixels), the joint-best homography success rate (0.999), and the highest throughput in the comparison, while carrying 1.28 million parameters against the twenty to thirty-four million of the others. Its median keypoint error is 1.45 pixels, which shows that the bulk of its detections are localised to within a pixel or two and that the mean is inflated by a small tail of harder frames; the same holds for TrackNet-court, whose median error is 1.44 pixels. The single metric on which MobileNetV3 does not lead is the mean reprojection error, where TrackNet-court is marginally better at 7.47 centimetres against 7.67, a difference too small to separate the two models in practice and far smaller than the gap to the remaining two.

TrackNet-court, the dataset's own baseline architecture, is the second strongest model and confirms that the task is well within reach of a heatmap detector: it reaches 0.988 accuracy at seven pixels and the best reprojection error, but it does so with sixteen times the parameters of MobileNetV3 and at the lowest throughput of the four. ResNet50 and HRNet, despite being the two largest and most capable backbones, are the two weakest court detectors at strict tolerance. ResNet50 reaches only 0.388 accuracy at five pixels and 0.688 at seven, and HRNet 0.460 and 0.849, both far below the stride-one models, and their mean reprojection errors of around thirty centimetres are four times worse. Their accuracy recovers sharply as the tolerance widens, to 0.960 and 0.986 at ten pixels and to near-parity with the others at twenty-five pixels, which indicates that they place each keypoint in the right neighbourhood but not to the pixel precision the strict thresholds demand. The reason for this pattern, which turns on the output resolution of the models rather than on the capacity of their backbones, is the subject of Section 5.6.

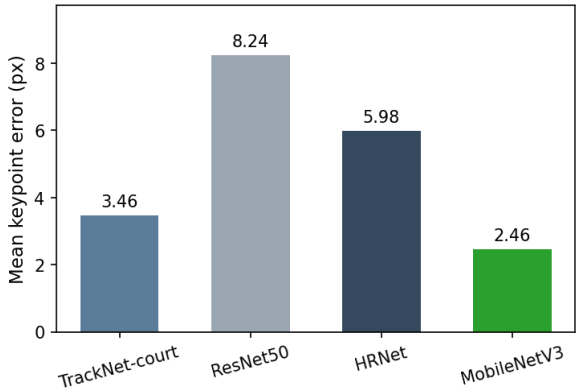
The per-keypoint breakdown of Figure 5.3 shows that no single landmark dominates the error: the two strong models are uniformly accurate across all fourteen keypoints, and the two weaker models are uniformly less accurate rather than failing on a particular



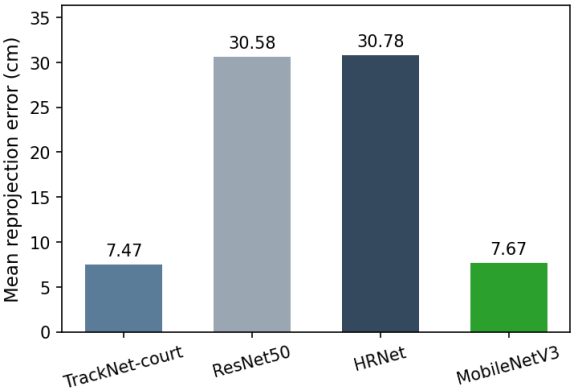
(a) PCK across the tolerance sweep.



(b) Accuracy against model size.



(c) Mean keypoint error.



(d) Mean court-plane reprojection error.

Figure 5.2: Four-model court-keypoint comparison; the plotted values are those of Table 5.3. (a) The percentage of correct keypoints across the $\{5, 7, 10, 25\}$-pixel tolerance sweep, where the two output-stride-one models (TrackNet-court and MobileNetV3) lead at every tolerance while the two stride-four models (ResNet50 and HRNet) start far lower at five pixels and converge towards the others only as the tolerance widens. (b) Accuracy at seven pixels against parameter count on a logarithmic axis, showing MobileNetV3 at the favourable corner, the smallest model at the top accuracy. (c) Mean keypoint error and (d) mean court-plane reprojection error per model, on both of which MobileNetV3 and TrackNet-court are far ahead of the two stride-four models.

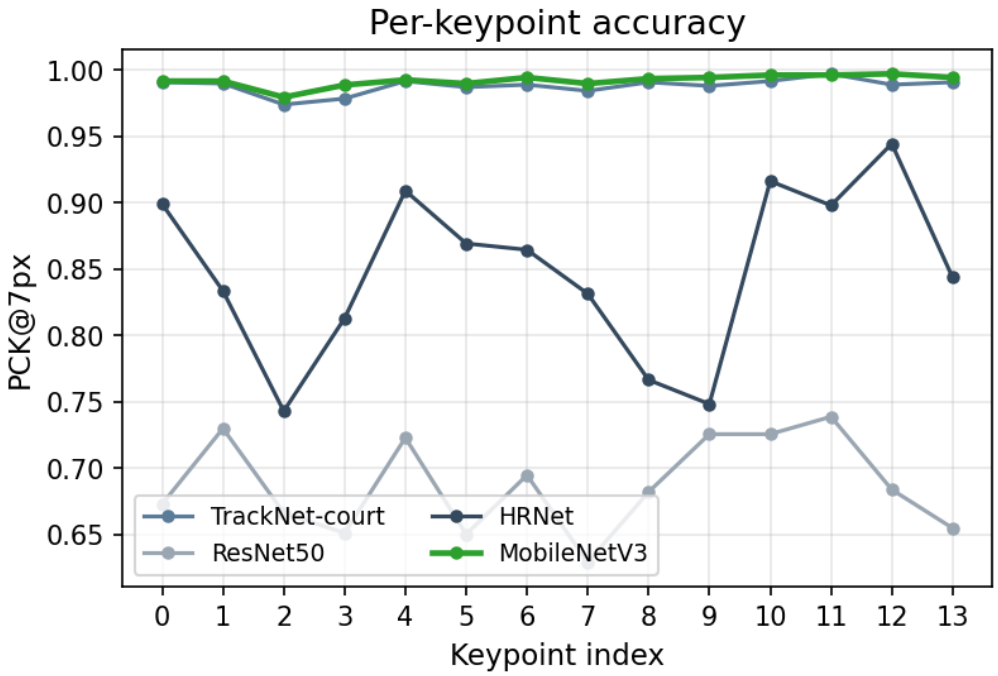


Figure 5.3: Accuracy at seven pixels for each of the fourteen keypoints. The two stride-one models are uniformly high across all landmarks, whereas the two stride-four models are lower across every landmark rather than failing on any particular one, which is consistent with a resolution-limited rather than a landmark-specific cause.

corner, which is consistent with a resolution-limited rather than a landmark-specific cause. The homography success rate is at least 0.995 for every model, so all four almost always produce enough well-supported keypoints to register the court; the models are separated by the precision of that registration, not by whether it can be obtained. Figure 5.4 shows a qualitative example of the selected model's output on a test frame, with the predicted keypoints close to the ground-truth annotations across the visible court.

These results answer RQ2. Among a heatmap baseline and three backbones of widely differing size, the best accuracy against latency trade-off is given not by the largest backbone but by the smallest, a 1.28-million-parameter MobileNetV3-Small with a full-resolution decoder, which is the most accurate model at every tolerance and the fastest in the comparison. Its mean court-plane reprojection error of 7.67 centimetres, obtained on essentially every test frame, is small relative to the scale of the court and the margins involved in a line call, so it is reliable enough to drive the homography that the integrated system of Chapter **??** depends on, and it is the model selected for that system.

5.6 Analysis

The central finding of this chapter is that strict-tolerance accuracy on this task is governed by the output resolution of the heatmap rather than by the capacity of the backbone, and the four models divide cleanly along that line. The two models that predict at output stride one, TrackNet-court and MobileNetV3, emit a heatmap at the full 360×640 input grid, so that a single heatmap cell corresponds to one input pixel and, after the factor-of-two rescaling to the original frame, to about two pixels in the space where accuracy is

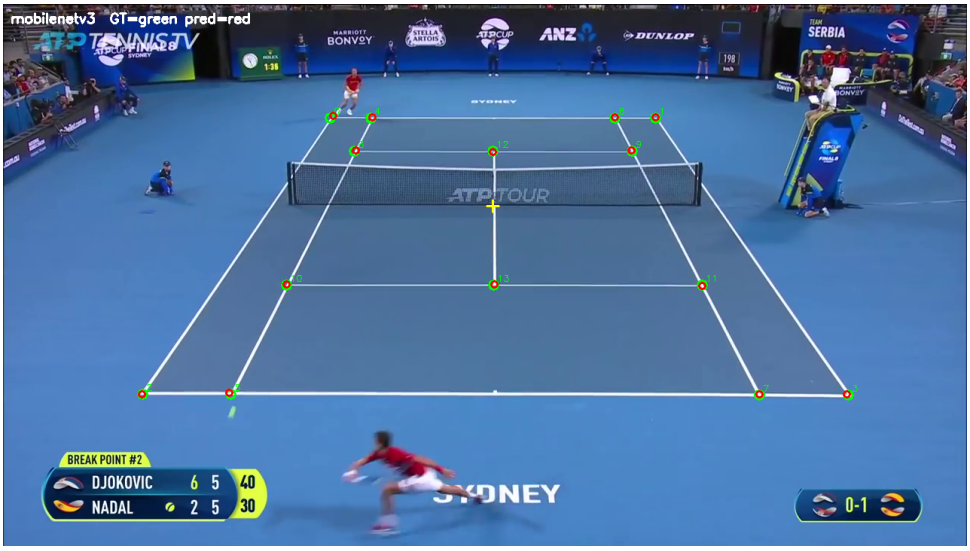


Figure 5.4: Qualitative output of the selected MobileNetV3 model on a court test frame. Ground-truth keypoints are shown in green, the model's predictions in red, and the predicted court centre as a yellow cross. The predicted landmarks coincide closely with the ground truth across the court, including the far baseline where perspective foreshortening is strongest.

measured. The two models that predict at output stride four, ResNet50 and HRNet, emit a 90×160 heatmap, so that one cell corresponds to four input pixels and to about eight pixels in the original frame. The peak of a coarse heatmap can therefore be no more precise than its cell, and although the sub-pixel refinement of Section 5.3 interpolates within a cell it cannot escape the underlying lattice. A five-pixel tolerance is smaller than the eight-pixel quantisation of the stride-four models, so even a perfectly centred prediction frequently falls outside it, which is why ResNet50 and HRNet score so low at five and seven pixels; once the tolerance widens past the quantisation step, at ten and twenty-five pixels, a correctly identified cell almost always falls within tolerance and the two models recover. The stride-one models, with a two-pixel quantisation, sit comfortably inside even the five-pixel tolerance, which is why they saturate the accuracy metric. The mean keypoint errors tell the same story in pixel terms, around six and eight pixels for the stride-four models against two and three for the stride-one models.

This explanation also resolves an apparent contradiction with the validation figures used during training. The metric monitored for early stopping and checkpoint selection was the accuracy at seven pixels measured in the resized 360×640 input space, and on that metric all four models reach comparable, high values, between 0.99 and 0.997. The seven-pixel tolerance in the input space corresponds to a fourteen-pixel tolerance in the original frame, which is wide enough to hide the quantisation penalty of the stride-four models; measuring the test accuracy in the original space at seven pixels, the stricter and more honest setting, is what exposes it. The lesson is methodological: the space in which a pixel tolerance is applied must be stated alongside the number, because the same nominal tolerance describes a different physical accuracy in the input space and in the original space, and the input-space monitoring metric flattered the coarse models.

It follows that the comparison is not a refutation of the larger backbones as feature extractors but of the particular stride-four heads used here, and in the case of HRNet of a head that does not exploit the backbone's defining high-resolution branch. A fairer

test of ResNet50 and HRNet would attach a decoder that restores the full resolution, as the U-Net decoder does for MobileNetV3, and their accuracy would be expected to rise accordingly. The practical conclusion for this thesis is nonetheless robust: a lightweight, full-resolution model already saturates the accuracy this task requires while costing a fraction of the parameters and running fastest, so there is no accuracy argument for paying the size and latency of the heavier models, and the small-backbone choice is the right one for a deployable system.

Two further points qualify the results. First, the mean reprojection error in centimetres should be read as an indicative geometric figure rather than an exact one, because of how it is computed: ground-truth keypoints are projected through the homography that was fitted to the predicted keypoints, so the number combines the accuracy of the keypoints with the quality of the homography fit in a single quantity, and a maximum reprojection error of well over a metre on the worst frames, even for the strong models, reflects the occasional frame on which a degenerate or poorly conditioned fit projects points far from their true positions. The success rate and the mean error together, rather than either alone, characterise the registration. Second, the court-centre channel that every model emits is supervised during training but is not scored at evaluation, because no ground-truth target for it is constructed in the evaluation loop; its accuracy is therefore reported as not evaluated, and the channel serves only as an auxiliary training signal and as an input to the geometric centre used elsewhere.

A local Hough-line refinement of the detected keypoints, which snaps each prediction towards the nearest painted court line, was also explored as an optional post-processing step. On the selected model it reduced the accuracy and the throughput while changing the reprojection error only marginally, so it offers no clear benefit at the operating point reached here and was not adopted for the integrated system. Finally, the six excluded test records noted in Section 5.1 are a reminder that public datasets carry annotation noise: they were found by a manual audit of the ground truth, and excluding them prevents a handful of geometrically impossible annotations from distorting the test metrics. Their identifiers are recorded in the project so that the exclusion is reproducible.

In summary, the court comparison answers RQ2 by selecting a 1.28-million-parameter MobileNetV3-Small model that is the most accurate of the four at every tolerance, registers the court to within about eight centimetres on essentially every frame, and runs fastest, and by showing that the accuracy ceiling on this task is set by the heatmap output resolution rather than by backbone capacity. The selected model provides the court geometry for the integrated system of Chapter **??**, where its homography projects the

tracked ball and the detected bounce into the court plane for the in-or-out line call.

Chapter 6

Bounce Event Detection

This chapter applies the protocol of Chapter 3 to the third perception sub-task, deciding at which frames the ball bounces on the court from the trajectory it traces through a rally. Where ball tracking locates the ball in each frame (Chapter 4) and court detection registers the playing surface (Chapter 5), bounce detection consumes the output of the former rather than the raw image: its input is a one-dimensional time series of ball positions, and its task is to mark the instant of ground contact within it. The sub-task runs on the TrackNet tennis dataset of Section 3.1, under the bounce-specific split of Section 3.4.1 that holds out match game7 as the test set, and it is scored by the event-detection metrics of Section 3.5.3. Three approaches of deliberately different sophistication are compared on a shared trajectory pipeline: a parameter-free heuristic scorer, a gradient-boosted per-frame model, and a small temporal convolutional network. The chapter answers the third research question (RQ3) of Section 1.3 on how these three approaches compare in accuracy, and on what each costs in engineering effort.

6.1 Task Formulation

A bounce is the frame at which the ball strikes the court, recorded in the dataset as status 2 in the annotation schema of Section 3.1.1. The task is posed as event detection in time rather than as per-frame classification, for the reason set out in Section 3.5.3: bounce frames make up only 2.6 % of the data (Table 3.2), so a per-frame accuracy would be dominated by the negative class and would reward a detector that simply never fires. What matters instead is the timing of the predicted events, measured by the event precision, recall, and F_1 at a temporal tolerance of $\pm k$ frames defined in Section 3.5.3, with the default tolerance $k = 3$.

Two properties of the problem shape every design choice that follows. The first is that a bounce is defined by how the ball moves, not by its appearance in a single frame, so the discriminative signal is kinematic. Because the image y axis grows downward, a ball falling towards the court has an increasing y coordinate, and the moment of ground contact is a local maximum of y followed by a reversal of the vertical velocity from positive (descending) to negative (ascending), accompanied by a sharp spike in the magnitude of the vertical acceleration. Figure 3.2 illustrates this signature, and the trajectory pipeline of Section 6.2 is built to expose it.

The second property is that a racket hit, status 1 in the same schema and almost as frequent as a bounce (516 against 523 events across the dataset, Table 3.2), produces a superficially similar kinematic event: it too reverses the ball's direction and spikes its

acceleration. Hits are therefore the natural hard negative for this task, and the central difficulty is to fire on bounces while staying silent on hits. The models are trained on a soft bounce-against-rest target with hits held as hard negatives, and the bounce-versus-hit confusion count of Section 3.5.3 measures how well each one keeps the two apart.

One scope limitation must be stated at the outset. The trajectory fed to every detector in this chapter is built from the ground-truth ball coordinates recorded in the dataset, not from the output of the ball tracker of Chapter 4. The reported scores are therefore an upper bound on the performance of a deployed system, in which the trajectory would carry the localisation error of the tracker, which is largest on exactly the occluded and near-ground frames where a bounce occurs. This decision isolates the bounce-detection question from the ball-tracking question, so that the three approaches are compared on the same clean signal, but the deployment gap is real and is taken up again in Section 6.6.

6.2 Trajectory Processing and Features

All three approaches share a single trajectory pipeline, so that the comparison varies only the scorer and keeps the preprocessing, the learning target, the decoding, and the metrics fixed. The pipeline operates strictly within each clip, so that no feature, window, or smoothing operation ever crosses a clip boundary where the ball identity and the time base are discontinuous.

Cleaning and interpolation. The raw trajectory is the sequence of annotated ball positions in a clip, sorted by frame index. Frames carrying the $(-1, -1)$ missing-ball sentinel of Section 3.1, whether from an invisible ball or a blank annotation, are converted to gaps. Short internal gaps of at most four frames are filled by linear interpolation, on the assumption that the ball follows a smooth path over so short an interval, and the interpolated frames are flagged so that a model can know which observations are inferred. Longer gaps and gaps at the start or end of a clip are left unfilled and are masked out of both the learning target and the loss, so that the detector is never trained or scored on a stretch of trajectory that does not exist.

Kinematics. The cleaned positions are differentiated to recover the velocity and acceleration that carry the bounce signature. On each contiguous run of valid frames of length at least seven, a Savitzky-Golay smoothing-differentiation filter with a seven-frame window and a quadratic polynomial estimates the first and second derivatives, which suppresses the per-frame jitter that a naive finite difference would amplify; shorter runs fall back to central differences. This yields the horizontal and vertical velocity and acceleration at every valid frame.

Features and target. Two feature representations are derived from the same cleaned trajectory, one for each kind of model. For the gradient-boosted model, which sees one frame at a time, each frame is described by a seventy-one-dimensional vector: eleven base quantities, namely the normalised position, the velocity and acceleration components, the speed and acceleration magnitude, the change in heading, the visibility, and the interpolation flag, together with sixty lag features that encode the local shape of the trajectory. The lag features compare each frame with its neighbours up to ten frames away in both directions, as signed vertical differences, absolute horizontal differences, and,

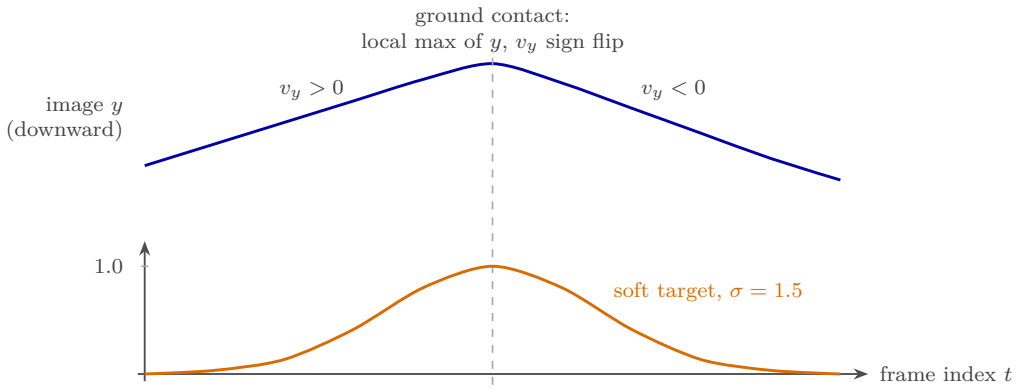


Figure 6.1: The kinematic signature of a bounce and its learning target, drawn schematically against the frame index. Upper curve: the vertical image coordinate y of the ball, which grows downward, so the ball descending towards the court raises y to a local maximum at ground contact and the vertical velocity v_y reverses from positive to negative there. Lower curve: the Gaussian-in-time soft target of unit peak and standard deviation 1.5 frames centred on the bounce frame, which the learned models regress. A racket hit produces a similar reversal and so must be separated from a bounce by cues other than the reversal alone.

most importantly, the scale-invariant ratios of the forward to the backward difference. These ratios are the principal discriminative feature: in free flight the trajectory is locally monotonic and the ratio of forward to backward motion is close to -1, whereas at a bounce the motion reverses and the ratio flips towards $+1$, a signature that is invariant to the ball's distance from the camera and so transfers across the perspective scale of the court. For the temporal convolutional network, which sees a window of frames at once and can learn such lag structure internally, each frame is described by only seven channels: the normalised position, the velocity and acceleration components, and the visibility.

The learning target is common to both learned models and is a soft one. Rather than a single positive frame per bounce, which would make an off-by-one prediction count as a complete miss and would offer almost no gradient on the scarce positive class, the target is a Gaussian bump in time of unit peak and a standard deviation of 1.5 frames, centred on each bounce frame. Hits and ordinary in-flight frames carry a target of zero, so that the soft label rewards a prediction near a true bounce in proportion to its temporal closeness while still treating hits as negatives. Figure 6.1 sketches the trajectory signature of a bounce and the soft target placed on it.

6.3 Models

Three scorers are compared, each mapping a clip's trajectory to a per-frame bounce score in $[0, 1]$ that the shared decoder of Section 6.4 turns into discrete events. They are chosen to span a deliberate range of sophistication and engineering cost, from a hand-built rule with no learned parameters to a trained neural network, which is the axis along which RQ3 asks them to be compared. Table 6.1 summarises them.

Heuristic. The heuristic scorer has no learned parameters. It combines three cues read directly from the kinematics, each normalised to $[0, 1]$ over the clip: the strength of a

Table 6.1: The three bounce-detection approaches. The input column distinguishes the per-frame models from the windowed one; the trained-parameters column counts only learned weights, excluding the single decode threshold that all three calibrate on the validation split.

Approach	Input per prediction	Discriminative mechanism	Trained params
Heuristic	one frame, with neighbours	three trajectory-shape cues, summed	none
GBM	one frame, 71 features	boosted regression trees	118 trees ($\leq 10^3$)
TCN	64-frame window, 7 channels	dilated temporal convolutions	10,433 weights

local maximum of the vertical position, the magnitude of a vertical velocity sign flip from descending to ascending within a small neighbourhood, and the magnitude of the vertical acceleration spike. The three cues are summed with equal weight, smoothed over a short temporal window, and renormalised to give the per-frame score. The only quantity fixed by data is the decode threshold, which is swept on the validation split to maximise the event F_1, so that the heuristic honours the same checkpoint-and-threshold contract as the learned models while itself learning nothing. It serves as an interpretable baseline that quantifies how far the bounce signature can be exploited by hand-coded rules alone.

Gradient-boosted model (GBM). The gradient-boosted model is an ensemble of regression trees, in the gradient boosting framework of Friedman [40], that maps the seventy-one-dimensional per-frame feature vector of Section 6.2 to the soft bounce target. The implementation prefers the CatBoost gradient-boosting library [42] and falls back to the histogram-based gradient-boosting regressor of scikit-learn [43] when it is unavailable, mirroring the import-or-fallback pattern used elsewhere in this work; the model evaluated here is the scikit-learn one. It is trained to minimise the squared error against the soft target, with the frames near a bounce up-weighted so that the scarce positive region is not drowned out by the negative class, for up to a thousand boosting iterations with early stopping on a held-out validation split, which halted it at 118 trees. Because it consumes one frame at a time, it relies on the explicit lag and ratio features to supply the temporal context that its trees cannot otherwise see.

Temporal convolutional network (TCN). The temporal convolutional network processes a window of sixty-four consecutive frames of the seven-channel representation and emits a bounce logit for every frame in the window, in the style of temporal convolutional models for sequence labelling [10, 39]. It is a small stack of four residual blocks of dilated one-dimensional convolutions, with the dilation doubling from one block to the next so that the receptive field grows exponentially and a single output frame can attend to its wider temporal neighbourhood, the same mechanism that lets dilated convolutions model long-range structure in sequence data [38]. The network is kept deliberately small, only 10,433 weights in the configuration used here, because the positive class is scarce and a larger model would overfit the few hundred training bounces. It is trained with the

Table 6.2: Bounce-detection results on the game7 test split (50 bounce events), at the default tolerance $k = 3$ frames and each model's validation-calibrated threshold. P, R, and F_1 are the event precision, recall, and F_1 at that tolerance; AP is the average precision over the threshold sweep; TP, FP, and FN are the event counts. Bold marks the best value in each column. The frames-per-second column is the throughput of the scorer alone on a precomputed trajectory and excludes ball tracking, so it is a relative ranking among the three approaches and is not comparable to the per-frame model speeds of Chapter 4.

Model	Thr.	F_1	P	R	AP	TP	FP	FN	FPS
Heuristic	0.20	0.578	0.459	0.780	0.394	39	46	11	**32,706**
GBM	0.60	**0.951**	**0.925**	**0.980**	0.874	49	**4**	**1**	21,216
TCN	0.60	0.925	0.875	**0.980**	**0.972**	49	7	**1**	4,703

Adam optimiser [47] against the soft target under a masked focal binary cross-entropy loss that down-weights the easy negative frames and up-weights the rare positives, with the loss masked out on the invalid frames identified by the cleaning step. At inference the network is run over overlapping windows whose predictions are averaged back into a single per-frame score for the clip. Unlike the gradient-boosted model it learns its own temporal features from the raw channels, at the cost of a training loop, a hardware accelerator to train on, and a heavier deployment artefact.

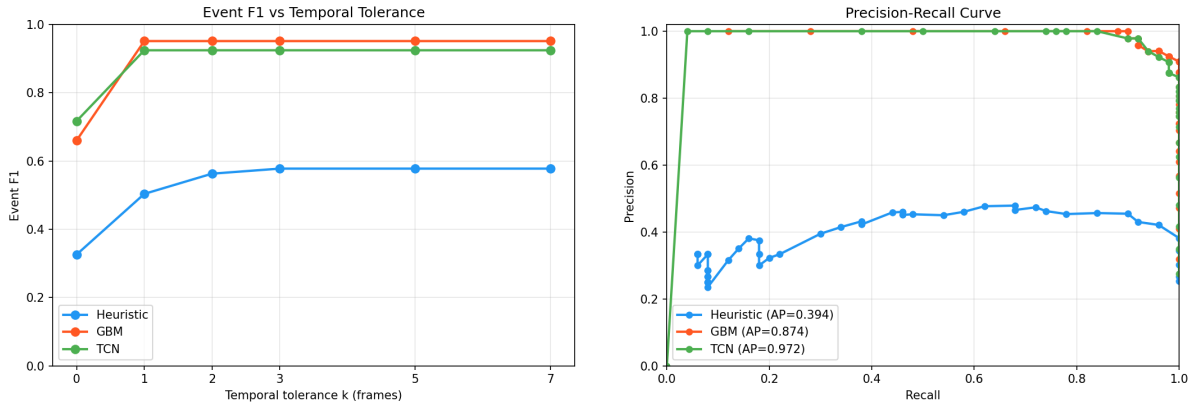
6.4 Decoding and Event Matching

To keep the comparison fair, the per-frame scores of all three models pass through one shared decoder and one shared matcher, so that any difference in the reported events is a difference between the scores and not between the post-processing. The decoder locates the peaks of the score signal that exceed the model's calibrated threshold, subject to a minimum spacing of five frames between successive peaks so that a single bounce is not reported twice, and treats each surviving peak as one predicted bounce event; frames masked invalid by the cleaning step are excluded from the search. The predicted events are then matched to the ground-truth bounce frames by the greedy one-to-one assignment within $\pm k$ frames defined in Section 3.5.3, closest pair first, from which the true positives, false positives, and false negatives, and hence the event precision, recall, and F_1 at tolerance k, follow. The default tolerance is $k = 3$ frames, and a sweep over $k \in \{0, 1, 2, 3, 5, 7\}$ reports how the score tightens or relaxes with the timing requirement.

6.5 Results

The three models were evaluated on the game7 test split, a single held-out match of 1 881 frames in nine clips containing 50 bounce events (Table 3.3). Each model's decode threshold was the value calibrated on the validation split. Table 6.2 reports the event metrics at the default tolerance together with the average precision and the event counts; Figure 6.2 plots the tolerance sweep and the precision-recall curves, and Figure 6.3 breaks the false positives down by what the model fired on.

The headline result is that both learned models are far stronger than the heuristic



(a) F_1 across the tolerance sweep.

(b) Precision-recall curves.

Figure 6.2: Bounce-detection comparison on the game7 test split. (a) Event F_1 as the matching tolerance k widens from 0 (exact frame) to 7 frames. At the exact frame the temporal network achieves the best F_1 (0.717 against the gradient-boosted model's 0.660 and the heuristic's 0.326), but from $k = 1$ onward the gradient-boosted model leads, and both learned models plateau by $k = 1$ while the heuristic continues to gain as the tolerance loosens. (b) The precision-recall curves traced by sweeping the decode threshold, whose areas are the average precisions of Table 6.2: the temporal network encloses the largest area (AP 0.972), the gradient-boosted model the next (0.874), and the heuristic far less (0.394).

and close to each other, but they lead on different measures. At the default tolerance the gradient-boosted model attains the best event F_1 of 0.951, recovering 49 of the 50 test bounces at a precision of 0.925, with only four false positives. The temporal network recovers the same 49 bounces but raises three more false alarms, for a precision of 0.875 and an F_1 of 0.925. The heuristic trails both substantially at an F_1 of 0.578: it finds most bounces, with a recall of 0.780, but at a precision of only 0.459, firing 46 false positives against 39 true ones.

The two learned models swap places depending on the measure, and the swap is informative rather than contradictory. On the average precision, which integrates over all decode thresholds and so measures the quality of the score ranking independently of any one operating point, the temporal network is clearly ahead at 0.972 against 0.874. On the exact-frame tolerance $k = 0$ in Figure 6.2a, the temporal network is again ahead, at 0.717 against 0.660, indicating that it places the peak on the true bounce frame more often. Yet at the chosen operating point and the default tolerance the gradient-boosted model wins on F_1, because at that single threshold it produces three fewer false positives on a test set of only fifty events, which is enough to separate two otherwise closely matched models. As Section 3.5.3 cautions, the average precision and the F_1 at a tolerance are different quantities at different operating points, and the honest reading is that the temporal network has the better score ranking and exact-frame timing while the gradient-boosted model is marginally more precise at the deployed threshold.

The bounce-versus-hit breakdown of Figure 6.3 explains where the heuristic's precision is lost. Of its 46 false positives, 31 are firings on racket hits and only 15 on neither event: the hand-coded cues cannot tell a bounce from a hit, because both reverse the ball's vertical motion and spike its acceleration, and the cues respond to the reversal itself. The learned models, which were trained with hits explicitly held as hard negatives, each

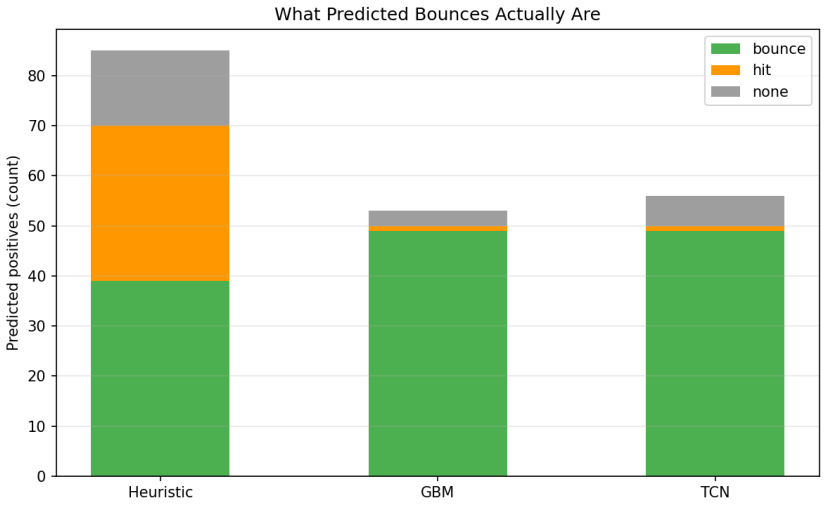


Figure 6.3: What each model fires on, bucketing every predicted bounce by whether the nearest ground-truth event within the tolerance window is a true bounce, a racket hit, or neither. The heuristic mistakes 31 racket hits for bounces, which is the majority of its false positives, because a hit produces a velocity reversal and acceleration spike much like a bounce. Both learned models, trained with hits as hard negatives, mistake only a single hit each, so almost all of their few false positives are firings on neither event.

mistake only a single hit for a bounce, so the hard-negative training is what closes the gap between the heuristic and the learned scorers. The remaining false positives of the learned models, three for the gradient-boosted model and six for the temporal network, fall on neither a bounce nor a hit, and on a fifty-event test set these few events are the entire margin between the two.

These results answer RQ3. A learned scorer is necessary: the parameter-free heuristic reaches only an F_1 of 0.578 and conflates hits with bounces, whereas both learned models exceed 0.92 and almost eliminate the hit confusion. Between the two learned models the difference is small and direction-dependent, the temporal network holding the better average precision and exact-frame timing and the gradient-boosted model the better operating-point F_1 and precision. The engineering costs that separate them, and the consequences of the ground-truth-trajectory ceiling, are the subject of the analysis below and inform the model carried into the integrated system of Chapter **??**.

6.6 Analysis

The first finding is that the bounce signature is learnable but not reducible to simple rules. The three cues the heuristic reads, the local maximum of the vertical position, the velocity sign flip, and the acceleration spike, are genuinely present at bounces, which is why the heuristic recovers most of them, but they are present at racket hits too, and a fixed weighted sum of them cannot resolve the ambiguity. The learned models succeed precisely where the heuristic fails, on the hit-against-bounce distinction, because they can exploit the finer differences in trajectory shape around the two kinds of event rather than responding to the reversal alone. This is the clearest demonstration in the bounce sub-task of the value of learning over hand engineering, and it parallels the finding of Chapter 4 that a learned localiser outperforms a geometric one.

The second finding concerns the choice between the two learned models, which the results deliberately leave close. The temporal convolutional network has the better precision-recall curve and locates the exact bounce frame more often, which suggests it is the stronger scorer in an absolute sense and would benefit most from a larger test set on which to tune its operating point. The gradient-boosted model matches it in recall, edges it in F_1 at the deployed threshold, and carries two further practical advantages. It needs no hardware accelerator, training in seconds on the engineered feature table, and it ships as a single serialised model file with no neural-network runtime, whereas the temporal network is best trained on a hardware accelerator, depends on a training loop and the windowing-and-averaging inference procedure, and would need a separate export step to deploy alongside the other neural components. For a system whose accuracy difference is within a handful of events on this test set, the deployment simplicity is the deciding factor, and the gradient-boosted model is the one selected for the integrated pipeline of Chapter **??**; the rationale is revisited there at the composition level.

The third point is the ceiling already flagged in Section 6.1. Every number in this chapter was obtained on a trajectory built from the ground-truth ball positions, so it measures the detectors against a clean signal and should be read as an upper bound. A deployed system would feed them the trajectory of the ball tracker of Chapter 4, which carries its largest localisation error on occluded and far balls, and a bounce occurs at exactly the moment the ball is low, small, and often partly hidden near the court surface. The kinematic cues that drive every model here are second-order derivatives of the position, which amplify any positional jitter, so the gap between these figures and deployed performance is expected to be real, although it is not measured in this work. The reported scores establish how well the timing problem can be solved given a good trajectory, not how well the end-to-end system bounces.

Two further caveats qualify the numbers. First, the test set is a single held-out match with only fifty bounce events, in keeping with the scope noted in Section 1.5, so each event is worth two points of recall and a difference of a few false positives moves the F_1 by several points; the gap between the two learned models in particular should not be over-read, and the per-game breakdown collapses to this one match. Second, the location of each bounce in court metres, which the integrated system uses for its line call, is not scored anywhere in this chapter, because the TrackNet dataset carries no court-keypoint ground truth and so no in-or-out label against which a projected bounce position could be checked; the court projection of a detected bounce is therefore a qualitative capability of the integrated system, taken up in Chapter **??**, and not a measured result.

In summary, the bounce sub-task answers RQ3 by showing that a learned per-frame model is needed to separate bounces from the hits that defeat a hand-coded heuristic, that a gradient-boosted model and a small temporal convolutional network are close competitors with complementary strengths, and that the gradient-boosted model is preferred for the integrated system on the strength of its operating-point precision and its markedly simpler deployment, all under an upper-bound protocol whose deployment gap

is acknowledged rather than measured.

Bibliography

[1] G. Thomas, R. Gade, T. B. Moeslund, P. Carr, and A. Hilton, "Computer vision for sports: Current applications and research topics," *Computer Vision and Image Understanding*, vol. 159, pp. 3–18, 2017.

[2] H. Collins and R. Evans, "You cannot be serious! Public understanding of technology with special reference to "Hawk-Eye"," *Public Understanding of Science*, vol. 17, no. 3, pp. 283–308, 2008.

[3] Y.-C. Huang, I.-N. Liao, C.-H. Chen, T.-U. İk, and W.-C. Peng, "TrackNet: A deep learning network for tracking high-speed and tiny objects in sports applications," in *Proc. 16th IEEE Int. Conf. Advanced Video and Signal Based Surveillance (AVSS)*, Taipei, Taiwan, 2019, pp. 1–8.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[5] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[7] K. Sun, B. Xiao, D. Liu, and J. Wang, "Deep high-resolution representation learning for human pose estimation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Long Beach, CA, USA, 2019, pp. 5693–5703.

[8] A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, Seoul, Korea, 2019, pp. 1314–1324.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785–794.

[10] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[11] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Columbus, OH, USA, 2014, pp. 580–587.

[12] R. Girshick, “Fast R-CNN,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1440–1448.

[13] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, 2017.

[14] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single shot MultiBox detector,” in *Proc. European Conf. Computer Vision (ECCV)*, Amsterdam, The Netherlands, 2016, pp. 21–37.

[15] J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” *arXiv preprint arXiv:1804.02767*, 2018.

[16] G. Jocher and J. Qiu, “Ultralytics YOLO11,” Software, version 11.0.0, 2024, https://github.com/ultralytics/ultralytics.

[17] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 2117–2125.

[18] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 2980–2988.

[19] G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng, X. Xie, and J. Han, “Towards large-scale small object detection: Survey and benchmarks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 13 467–13 488, 2023.

[20] S.-E. Wei, V. Ramakrishna, T. Kanade, and Y. Sheikh, “Convolutional pose machines,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 4724–4732.

[21] A. Newell, K. Yang, and J. Deng, “Stacked hourglass networks for human pose estimation,” in *Proc. European Conf. Computer Vision (ECCV)*, Amsterdam, The Netherlands, 2016, pp. 483–499.

[22] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as points,” *arXiv preprint arXiv:1904.07850*, 2019.

[23] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, Phoenix, AZ, USA, 2016, pp. 3464–3468.

[24] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, Beijing, China, 2017, pp. 3645–3649.

[25] N.-E. Sun, Y.-C. Lin, S.-P. Chuang, T.-H. Hsu, D.-R. Yu, H.-Y. Chung, and T.-U. Ik, “TrackNetV2: Efficient shuttlecock tracking network,” in *Proc. Int. Conf. Pervasive Artificial Intelligence (ICPAI)*, Taipei, Taiwan, 2020, pp. 86–91.

[26] Y.-J. Chen and Y.-S. Wang, "TrackNetV3: Enhancing shuttlecock tracking with augmentations and trajectory rectification," in *Proc. 5th ACM Int. Conf. Multimedia in Asia (MMAsia)*, Tainan, Taiwan, 2023.

[27] A. Raj, L. Wang, and T. Gedeon, "TrackNetV4: Enhancing fast sports object tracking with motion attention maps," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, Hyderabad, India, 2025, arXiv:2409.14543.

[28] H. Tang, Y. Chen, L. Jiang, Q. Li, and X. Guo, "TrackNetV5: Residual-driven spatio-temporal refinement and motion direction decoupling for fast object tracking," 2025. [Online]. Available: https://arxiv.org/abs/2512.02789

[29] R. Voeikov, N. Falaleev, and R. Baikulov, "TTNet: Real-time temporal and spatial video analysis of table tennis," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, USA, 2020, pp. 884–885.

[30] V. Renò, N. Mosca, R. Marani, M. Nitti, T. D'Orazio, and E. Stella, "Convolutional neural networks based ball detection in tennis games," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, Salt Lake City, UT, USA, 2018, pp. 1758–1764.

[31] B. Xiao, H. Wu, and Y. Wei, "Simple baselines for human pose estimation and tracking," in *Proc. European Conf. Computer Vision (ECCV)*, Munich, Germany, 2018, pp. 472–487.

[32] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.

[33] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[34] E. Brachmann, A. Krull, S. Nowozin, J. Shotton, F. Michel, S. Gumhold, and C. Rother, "DSAC: Differentiable RANSAC for camera localization," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 6684–6692.

[35] N. Homayounfar, S. Fidler, and R. Urtasun, "Sports field localization via deep structured models," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 5212–5220.

[36] W. Jiang, J. C. G. Higuera, B. Angles, W. Sun, M. Javan, and K. M. Yi, "Optimizing through learned errors for accurate sports field registration," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, Snowmass Village, CO, USA, 2020, pp. 201–210.

[37] X. Nie, S. Chen, and R. Hamid, "A robust and efficient framework for sports-field registration," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, Waikoloa, HI, USA, 2021, pp. 1936–1944.

[38] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves *et al.*, "WaveNet: A generative model for raw audio," *arXiv preprint arXiv:1609.03499*, 2016.

[39] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal convolutional networks for action segmentation and detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 1003–1012.

[40] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[41] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NIPS)*, Long Beach, CA, USA, 2017, pp. 3146–3154.

[42] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems (NeurIPS)*, Montréal, Canada, 2018, pp. 6639–6649.

[43] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[44] N. Owens, C. Harris, and C. Stennett, "Hawk-eye tennis system," in *Proc. Int. Conf. Visual Information Engineering (VIE)*, Guildford, U.K., 2003, pp. 182–185.

[45] Z. Zhao, W. Chai, S. Hao, W. Hu, G. Wang, S. Cao, M. Song, J.-N. Hwang, and G. Wang, "A survey of deep learning in sports applications: Perception, comprehension, and decision," *arXiv preprint arXiv:2307.03353*, 2023.

[46] yastrebksv, "TennisCourtDetector: Deep learning network for detecting tennis court," Software and dataset, 2023, https://github.com/yastrebksv/TennisCourtDetector (accessed Jun. 2026).

[47] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.